CAMPUS360 ACADEMIC PLATFORM

Complete Source Code & Architecture Specification

Attendance Tracker • Syllabus Tracker • SGPA Calculator • CGPA Planner • REST API

Branch: shiffana

Repository: https://github.com/HaydenVictor420/Campus360.git

Local Endpoint: http://localhost:3000

Runtime: Node.js (v24 LTS) / Express.js

Generated Date: November 4, 2026

Confidential & Educational Project Submission — Campus360 Module

Table of Contents

1. server.js	Backend REST API (Node.js/Express)
2. package.json	Project Manifest & Scripts
3. public/index.html	Frontend Dashboard & SPA View (HTML5)
4. public/css/style.css	Complete UI Design System & Responsive Styles (CSS3)
5. public/js/app.js	App Controller, Sync Engine & Modal Manager (JavaScript)
6. public/js/attendance.js	Attendance Tracker, Safe Bunk Calculator & CSV Importer
7. public/js/syllabus.js	Syllabus Completion & Topic Tracker (JavaScript)
8. public/js/gpa.js	Semester GPA (SGPA) Dynamic Calculator (JavaScript)
9. public/js/cgpa.js	Cumulative CGPA Calculator & Target Planner (JavaScript)
10. public/js/settings.js	Institutional Rules, Grading Scales & Data Backups
11. data/database.json	Sample Dataset & JSON Database Schema
12. README.md	System Architecture & Mathematical Formulas Documentation

1. server.js

Backend REST API (Node.js/Express) (851 lines)

```
const express = require('express');
const cors = require('cors');
const fs = require('fs');
const path = require('path');

const app = express();
const PORT = process.env.PORT || 3000;
const DB_FILE = path.join(__dirname, 'data', 'database.json');

app.use(cors());
app.use(express.json({ limit: '10mb' }));
app.use(express.urlencoded({ extended: true }));
app.use(express.static(path.join(__dirname, 'public')));

// Helpers for Database read/write
function readDatabase() {
  try {
    if (!fs.existsSync(DB_FILE)) {
      throw new Error('Database file does not exist');
    }
    const data = fs.readFileSync(DB_FILE, 'utf8');
    return JSON.parse(data);
  } catch (err) {
    console.error('Error reading database:', err);
    return {
      settings: { minAttendanceThreshold: 75, warningMargin: 5, institutionRule: 'weighted_credits', gradeScale: [] },
      attendance: [],
      syllabus: [],
      gpaCourses: [],
      cgpaSemesters: []
    };
  }
}

function writeDatabase(data) {
  try {
    fs.writeFileSync(DB_FILE, JSON.stringify(data, null, 2), 'utf8');
    return true;
  } catch (err) {
    console.error('Error writing database:', err);
    return false;
  }
}

// Attendance Calculation Helper
function enrichAttendanceRecord(rec, threshold, margin) {
  const conducted = parseInt(rec.conducted, 10) || 0;
  const attended = parseInt(rec.attended, 10) || 0;
  const t = parseFloat(threshold) || 75;
  const m = parseFloat(margin) || 5;

  let percentage = null;
  let status = 'none'; // 'safe' | 'warning' | 'danger' | 'none'
  let classesNeeded = 0;
  let safeBunks = 0;

  if (conducted > 0) {
    percentage = Math.round((attended / conducted) * 10000) / 100;
    if (percentage >= t) {
      status = 'safe';
      // Calculate how many subsequent classes student can miss safely:
      // (attended) / (conducted + m) >= t / 100
      // m <= (100 * attended - t * conducted) / t
      const maxBunks = Math.floor((100 * attended - t * conducted) / t);
      safeBunks = Math.max(0, maxBunks);
      classesNeeded = 0;
    } else {
      if (percentage >= t - m) {
        status = 'warning';
      } else {
        status = 'danger';
      }
      // Calculate how many consecutive classes must be attended to reach threshold:
```

```
      // (attended + n) / (conducted + n) >= t / 100
      // 100*(attended + n) >= t*(conducted + n)
      // n*(100 - t) >= t*conducted - 100*attended
      if (t < 100) {
        const needed = Math.ceil((t * conducted - 100 * attended) / (100 - t));
        classesNeeded = Math.max(0, needed);
      } else {
        classesNeeded = conducted - attended;
      }
      safeBunks = 0;
    }
  }

  return {
    ...rec,
    conducted,
    attended,
    percentage,
    status,
    classesNeeded,
    safeBunks,
    threshold: t
  };
}

// Syllabus Calculation Helper
function enrichSyllabusRecord(item) {
  const units = item.units || [];
  let totalTopics = 0;
  let completedTopics = 0;

  const enrichedUnits = units.map(u => {
    const tot = parseInt(u.topicsTotal, 10) || 0;
    const comp = parseInt(u.topicsCompleted, 10) || 0;
    totalTopics += tot;
    completedTopics += comp;
    const unitPct = tot > 0 ? Math.round((comp / tot) * 100) : 0;
    let unitStatus = u.status;
    if (unitPct === 100) unitStatus = 'Completed';
    else if (unitPct > 0) unitStatus = 'In Progress';
    else unitStatus = 'Not Started';

    return {
      ...u,
      topicsTotal: tot,
      topicsCompleted: comp,
      percentage: unitPct,
      status: unitStatus
    };
  });

  const percentage = totalTopics > 0 ? Math.round((completedTopics / totalTopics) * 100) : 0;
  let status = 'Not Started';
  if (percentage === 100) status = 'Completed';
  else if (percentage >= 60) status = 'On Track';
  else if (percentage > 0) status = 'In Progress';

  return {
    ...item,
    totalTopics,
    completedTopics,
    percentage,
    status,
    units: enrichedUnits
  };
}

// ==================== API ROUTES ====================

// Health Check
app.get('/api/health', (req, res) => {
  res.json({ status: 'ok', timestamp: new Date().toISOString(), service: 'Campus360 Tracker' });
});

// Full Dashboard Stats
app.get('/api/dashboard', (req, res) => {
  const db = readDatabase();
  const threshold = db.settings?.minAttendanceThreshold || 75;
```

```
  const margin = db.settings?.warningMargin || 5;

  // Attendance stats
  let totalConducted = 0;
  let totalAttended = 0;
  let safeCount = 0;
  let warningCount = 0;
  let dangerCount = 0;
  const attendanceAlerts = [];

  const attendanceList = (db.attendance || []).map(item => {
    const enriched = enrichAttendanceRecord(item, threshold, margin);
    totalConducted += enriched.conducted;
    totalAttended += enriched.attended;

    if (enriched.status === 'safe') safeCount++;
    else if (enriched.status === 'warning') {
      warningCount++;
      attendanceAlerts.push({
        type: 'warning',
        subject: enriched.name,
        code: enriched.code,
        percentage: enriched.percentage,
        message: `Attendance is near threshold margin (${enriched.percentage}%). Need
${enriched.classesNeeded} more class(es).`
      });
    } else if (enriched.status === 'danger') {
      dangerCount++;
      attendanceAlerts.push({
        type: 'danger',
        subject: enriched.name,
        code: enriched.code,
        percentage: enriched.percentage,
        message: `Attendance shortage (${enriched.percentage}% < ${threshold}%). Must attend next
${enriched.classesNeeded} consecutive class(es).`
      });
    }
    return enriched;
  });

  const overallAttendance = totalConducted > 0
    ? Math.round((totalAttended / totalConducted) * 10000) / 100
    : 0;

  // GPA calculation
  let totalGpaCredits = 0;
  let totalGpaPointsWeighted = 0;
  (db.gpaCourses || []).forEach(c => {
    const creds = parseFloat(c.credits) || 0;
    const pts = parseFloat(c.points) || 0;
    totalGpaCredits += creds;
    totalGpaPointsWeighted += (creds * pts);
  });
  const currentSemesterGPA = totalGpaCredits > 0
    ? Math.round((totalGpaPointsWeighted / totalGpaCredits) * 100) / 100
    : 0;

  // CGPA calculation
  let totalCgpaCredits = 0;
  let totalCgpaPointsWeighted = 0;
  (db.cgpaSemesters || []).forEach(s => {
    const creds = parseFloat(s.credits) || 0;
    const sgpa = parseFloat(s.sgpa) || 0;
    totalCgpaCredits += creds;
    totalCgpaPointsWeighted += (creds * sgpa);
  });
  const cumulativeCGPA = totalCgpaCredits > 0
    ? Math.round((totalCgpaPointsWeighted / totalCgpaCredits) * 100) / 100
    : 0;

  // Syllabus stats
  let totalSyllabusTopics = 0;
  let completedSyllabusTopics = 0;
  const syllabusList = (db.syllabus || []).map(enrichSyllabusRecord);
  syllabusList.forEach(s => {
    totalSyllabusTopics += s.totalTopics;
    completedSyllabusTopics += s.completedTopics;
  });
  const overallSyllabusPercentage = totalSyllabusTopics > 0
    ? Math.round((completedSyllabusTopics / totalSyllabusTopics) * 100)
```

```
    : 0;

  res.json({
    summary: {
      overallAttendance,
      totalConducted,
      totalAttended,
      totalSubjects: attendanceList.length,
      safeSubjects: safeCount,
      warningSubjects: warningCount,
      dangerSubjects: dangerCount,
      currentSemesterGPA,
      totalGpaCredits,
      cumulativeCGPA,
      totalCgpaCredits,
      overallSyllabusPercentage,
      totalSyllabusTopics,
      completedSyllabusTopics
    },
    alerts: attendanceAlerts,
    attendanceOverview: attendanceList,
    syllabusOverview: syllabusList.map(s => ({
      id: s.id,
      subjectCode: s.subjectCode,
      subjectName: s.subjectName,
      percentage: s.percentage,
      status: s.status,
      completedTopics: s.completedTopics,
      totalTopics: s.totalTopics
    }))
  });
});

// ==================== ATTENDANCE ENDPOINTS ====================

app.get('/api/attendance', (req, res) => {
  const db = readDatabase();
  const threshold = db.settings?.minAttendanceThreshold || 75;
  const margin = db.settings?.warningMargin || 5;
  const records = (db.attendance || []).map(r => enrichAttendanceRecord(r, threshold, margin));
  res.json(records);
});

app.post('/api/attendance', (req, res) => {
  const { code, name, conducted, attended, faculty } = req.body;

  if (!name || name.trim() === '') {
    return res.status(400).json({ error: 'Subject name is required.' });
  }

  const cond = parseInt(conducted, 10);
  const att = parseInt(attended, 10);

  if (isNaN(cond) || cond < 0 || isNaN(att) || att < 0) {
    return res.status(400).json({ error: 'Conducted and attended classes must be non-negative numbers.' });
  }

  if (att > cond) {
    return res.status(400).json({ error: 'Attended classes cannot exceed total conducted classes.' });
  }

  const db = readDatabase();
  const newRecord = {
    id: 'att_' + Date.now(),
    code: (code || '').trim() || 'SUB-' + (db.attendance.length + 1),
    name: name.trim(),
    conducted: cond,
    attended: att,
    faculty: (faculty || '').trim(),
    lastUpdated: new Date().toISOString().split('T')[0]
  };

  db.attendance.push(newRecord);

  // Also initialize syllabus tracker for this subject if not exists
  const existingSyllabus = (db.syllabus || []).find(s => s.subjectCode === newRecord.code);
  if (!existingSyllabus) {
    db.syllabus.push({
```

```
      id: 'syl_' + Date.now(),
      subjectCode: newRecord.code,
      subjectName: newRecord.name,
      totalUnits: 5,
      units: [
        { unitNo: 1, title: 'Unit 1: Fundamentals', topicsTotal: 5, topicsCompleted: 0, status: 'Not Started' },
        { unitNo: 2, title: 'Unit 2: Core Concepts', topicsTotal: 5, topicsCompleted: 0, status: 'Not Started' },
        { unitNo: 3, title: 'Unit 3: Intermediate Topics', topicsTotal: 5, topicsCompleted: 0, status: 'Not Started' },
        { unitNo: 4, title: 'Unit 4: Advanced Systems', topicsTotal: 5, topicsCompleted: 0, status: 'Not Started' },
        { unitNo: 5, title: 'Unit 5: Applications & Case Studies', topicsTotal: 5, topicsCompleted: 0, status: 'Not Started' }
      ]
    });
  }

  writeDatabase(db);
  const enriched = enrichAttendanceRecord(newRecord, db.settings.minAttendanceThreshold, db.settings.warningMargin);
  res.status(201).json(enriched);
});

app.put('/api/attendance/:id', (req, res) => {
  const { id } = req.params;
  const { code, name, conducted, attended, faculty } = req.body;

  const db = readDatabase();
  const index = db.attendance.findIndex(a => a.id === id);
  if (index === -1) {
    return res.status(404).json({ error: 'Attendance record not found.' });
  }

  const cond = parseInt(conducted, 10);
  const att = parseInt(attended, 10);

  if (isNaN(cond) || cond < 0 || isNaN(att) || att < 0) {
    return res.status(400).json({ error: 'Conducted and attended classes must be non-negative numbers.' });
  }
  if (att > cond) {
    return res.status(400).json({ error: 'Attended classes cannot exceed total conducted classes.' });
  }

  const oldCode = db.attendance[index].code;
  db.attendance[index] = {
    ...db.attendance[index],
    code: (code || db.attendance[index].code).trim(),
    name: (name || db.attendance[index].name).trim(),
    conducted: cond,
    attended: att,
    faculty: faculty !== undefined ? faculty.trim() : db.attendance[index].faculty,
    lastUpdated: new Date().toISOString().split('T')[0]
  };

  // Sync subject name/code in syllabus if changed
  const syl = (db.syllabus || []).find(s => s.subjectCode === oldCode);
  if (syl) {
    syl.subjectCode = db.attendance[index].code;
    syl.subjectName = db.attendance[index].name;
  }

  writeDatabase(db);
  const enriched = enrichAttendanceRecord(db.attendance[index], db.settings.minAttendanceThreshold, db.settings.warningMargin);
  res.json(enriched);
});

app.delete('/api/attendance/:id', (req, res) => {
  const { id } = req.params;
  const db = readDatabase();
  const index = db.attendance.findIndex(a => a.id === id);
  if (index === -1) {
    return res.status(404).json({ error: 'Attendance record not found.' });
  }

  const removed = db.attendance.splice(index, 1)[0];
  writeDatabase(db);
  res.json({ message: 'Subject removed successfully', removed });
});

// Quick increment (+1 attended, or +1 missed)
app.post('/api/attendance/:id/increment', (req, res) => {
  const { id } = req.params;
```

```
  const { action } = req.body; // 'present' or 'absent'

  const db = readDatabase();
  const rec = db.attendance.find(a => a.id === id);
  if (!rec) {
    return res.status(404).json({ error: 'Attendance record not found.' });
  }

  rec.conducted = (rec.conducted || 0) + 1;
  if (action === 'present') {
    rec.attended = (rec.attended || 0) + 1;
  }
  rec.lastUpdated = new Date().toISOString().split('T')[0];

  writeDatabase(db);
  const enriched = enrichAttendanceRecord(rec, db.settings.minAttendanceThreshold,
db.settings.warningMargin);
  res.json(enriched);
});

// Bulk Import Attendance
app.post('/api/attendance/bulk', (req, res) => {
  const { items, replaceExisting } = req.body;
  if (!Array.isArray(items) || items.length === 0) {
    return res.status(400).json({ error: 'Invalid or empty bulk import dataset.' });
  }

  const errors = [];
  const validItems = [];

  items.forEach((item, idx) => {
    const rowNum = idx + 1;
    const name = item.name || item.subject || item.Subject;
    const code = item.code || item.subjectCode || item.Code || `SUB-${rowNum}`;
    const conducted = parseInt(item.conducted || item.classesHeld || item.classes_conducted ||
item.Conducted, 10);
    const attended = parseInt(item.attended || item.classesAttended || item.classes_attended ||
item.Attended, 10);
    if (!name || name.toString().trim() === '') {
      errors.push(`Row ${rowNum}: Subject name is missing.`);
      return;
    }
    if (isNaN(conducted) || conducted < 0) {
      errors.push(`Row ${rowNum} (${name}): Invalid conducted classes.`);
      return;
    }
    if (isNaN(attended) || attended < 0) {
      errors.push(`Row ${rowNum} (${name}): Invalid attended classes.`);
      return;
    }
    if (attended > conducted) {
      errors.push(`Row ${rowNum} (${name}): Attended classes (${attended}) exceed conducted
(${conducted}).`);
      return;
    }

    validItems.push({
      id: 'att_' + Date.now() + '_' + idx,
      code: code.toString().trim(),
      name: name.toString().trim(),
      conducted,
      attended,
      faculty: (item.faculty || '').toString().trim(),
      lastUpdated: new Date().toISOString().split('T')[0]
    });
  });

  if (validItems.length === 0) {
    return res.status(400).json({ error: 'No valid records found in the import.', details: errors });
  }

  const db = readDatabase();
  if (replaceExisting) {
    db.attendance = validItems;
  } else {
    // Merge or append
    validItems.forEach(newItem => {
      const existingIdx = db.attendance.findIndex(a => a.name.toLowerCase() === newItem.name.toLowerCase()
|| a.code.toLowerCase() === newItem.code.toLowerCase());
      if (existingIdx !== -1) {
        db.attendance[existingIdx] = newItem;
      } else {
```

```
        db.attendance.push(newItem);
      }
    });
  }

  writeDatabase(db);
  res.json({
    message: `Successfully imported ${validItems.length} records.`,
    count: validItems.length,
    errors: errors.length > 0 ? errors : undefined
  });
});

// ==================== SYLLABUS ENDPOINTS ====================

app.get('/api/syllabus', (req, res) => {
  const db = readDatabase();
  const list = (db.syllabus || []).map(enrichSyllabusRecord);
  res.json(list);
});

app.post('/api/syllabus', (req, res) => {
  const { subjectCode, subjectName, totalUnits, units } = req.body;
  if (!subjectName) {
    return res.status(400).json({ error: 'Subject name is required.' });
  }

  const db = readDatabase();
  const newSyl = {
    id: 'syl_' + Date.now(),
    subjectCode: (subjectCode || 'SUB-' + (db.syllabus.length + 1)).trim(),
    subjectName: subjectName.trim(),
    totalUnits: parseInt(totalUnits, 10) || (units ? units.length : 5),
    units: units || [
      { unitNo: 1, title: 'Unit 1: Introduction', topicsTotal: 5, topicsCompleted: 0, status: 'Not
Started' }
    ]
  };

  db.syllabus.push(newSyl);
  writeDatabase(db);
  res.status(201).json(enrichSyllabusRecord(newSyl));
});

app.put('/api/syllabus/:id', (req, res) => {
  const { id } = req.params;
  const db = readDatabase();
  const index = db.syllabus.findIndex(s => s.id === id);
  if (index === -1) {
    return res.status(404).json({ error: 'Syllabus item not found.' });
  }

  const body = req.body;
  db.syllabus[index] = {
    ...db.syllabus[index],
    ...body,
    id: db.syllabus[index].id
  };

  writeDatabase(db);
  res.json(enrichSyllabusRecord(db.syllabus[index]));
});

// Update single unit within a syllabus
app.put('/api/syllabus/:id/unit/:unitNo', (req, res) => {
  const { id, unitNo } = req.params;
  const { title, topicsTotal, topicsCompleted, status } = req.body;

  const db = readDatabase();
  const syl = db.syllabus.find(s => s.id === id);
  if (!syl) {
    return res.status(404).json({ error: 'Syllabus entry not found.' });
  }

  const unit = syl.units.find(u => u.unitNo === parseInt(unitNo, 10));
  if (!unit) {
    return res.status(404).json({ error: 'Unit not found.' });
  }

```

```
  if (title !== undefined) unit.title = title.trim();
  if (topicsTotal !== undefined) unit.topicsTotal = Math.max(0, parseInt(topicsTotal, 10) || 0);
  if (topicsCompleted !== undefined) {
    const comp = Math.max(0, parseInt(topicsCompleted, 10) || 0);
    unit.topicsCompleted = Math.min(comp, unit.topicsTotal);
  }
  if (status !== undefined) unit.status = status;

  writeDatabase(db);
  res.json(enrichSyllabusRecord(syl));
});

app.delete('/api/syllabus/:id', (req, res) => {
  const { id } = req.params;
  const db = readDatabase();
  const index = db.syllabus.findIndex(s => s.id === id);
  if (index === -1) {
    return res.status(404).json({ error: 'Syllabus item not found.' });
  }

  const removed = db.syllabus.splice(index, 1)[0];
  writeDatabase(db);
  res.json({ message: 'Syllabus item removed', removed });
});

// ==================== GPA ENDPOINTS ====================

app.get('/api/gpa', (req, res) => {
  const db = readDatabase();
  const courses = db.gpaCourses || [];
  let totalCredits = 0;
  let weightedPoints = 0;

  courses.forEach(c => {
    const cred = parseFloat(c.credits) || 0;
    const pt = parseFloat(c.points) || 0;
    totalCredits += cred;
    weightedPoints += (cred * pt);
  });

  const gpa = totalCredits > 0 ? Math.round((weightedPoints / totalCredits) * 100) / 100 : 0;
  res.json({
    courses,
    totalCredits,
    weightedPoints,
    gpa,
    formula: 'GPA = :2„7&VF—G2 r w& FR  ö–çG2’ ò £(Credits)'
  });
});

app.post('/api/gpa', (req, res) => {
  const { courses } = req.body;
  if (!Array.isArray(courses)) {
    return res.status(400).json({ error: 'Courses must be an array.' });
  }

  const db = readDatabase();
  db.gpaCourses = courses.map((c, idx) => ({
    id: c.id || 'gpa_' + Date.now() + '_' + idx,
    code: (c.code || '').trim(),
    name: (c.name || 'Subject ' + (idx + 1)).trim(),
    credits: Math.max(0, parseFloat(c.credits) || 0),
    grade: (c.grade || '').trim().toUpperCase(),
    points: Math.max(0, parseFloat(c.points) || 0)
  }));

  writeDatabase(db);
  res.json({ message: 'GPA courses saved', count: db.gpaCourses.length });
});

// ==================== CGPA ENDPOINTS ====================

app.get('/api/cgpa', (req, res) => {
  const db = readDatabase();
  const semesters = db.cgpaSemesters || [];
  let totalCredits = 0;
  let weightedSgpa = 0;

```

```
  semesters.forEach(s => {
    const cred = parseFloat(s.credits) || 0;
    const sgpa = parseFloat(s.sgpa) || 0;
    totalCredits += cred;
    weightedSgpa += (cred * sgpa);
  });

  const cgpa = totalCredits > 0 ? Math.round((weightedSgpa / totalCredits) * 100) / 100 : 0;
  res.json({
    semesters,
    totalCredits,
    weightedSgpa,
    cgpa,
    formula: 'CGPA = :2…6VÖW7FW" 7&VF—G2 r 4u  ’ ò £(Semester Credits)'
  });
});

app.post('/api/cgpa', (req, res) => {
  const { semesters } = req.body;
  if (!Array.isArray(semesters)) {
    return res.status(400).json({ error: 'Semesters must be an array.' });
  }

  const db = readDatabase();
  db.cgpaSemesters = semesters.map((s, idx) => ({
    id: s.id || 'sem_' + Date.now() + '_' + idx,
    semester: (s.semester || 'Semester ' + (idx + 1)).trim(),
    credits: Math.max(0, parseFloat(s.credits) || 0),
    sgpa: Math.max(0, parseFloat(s.sgpa) || 0)
  }));

  writeDatabase(db);
  res.json({ message: 'CGPA semesters saved', count: db.cgpaSemesters.length });
});

// ==================== SETTINGS ENDPOINTS ====================

app.get('/api/settings', (req, res) => {
  const db = readDatabase();
  res.json(db.settings || {});
});

app.post('/api/settings', (req, res) => {
  const { minAttendanceThreshold, warningMargin, institutionRule, gradeScale } = req.body;
  const db = readDatabase();

  if (minAttendanceThreshold !== undefined) {
    const th = parseFloat(minAttendanceThreshold);
    if (isNaN(th) || th <= 0 || th > 100) {
      return res.status(400).json({ error: 'Attendance threshold must be between 1 and 100.' });
    }
    db.settings.minAttendanceThreshold = th;
  }

  if (warningMargin !== undefined) {
    const wm = parseFloat(warningMargin);
    if (!isNaN(wm) && wm >= 0 && wm < 50) {
      db.settings.warningMargin = wm;
    }
  }

  if (institutionRule) {
    db.settings.institutionRule = institutionRule;
  }

  if (Array.isArray(gradeScale) && gradeScale.length > 0) {
    db.settings.gradeScale = gradeScale;
  }

  writeDatabase(db);
  res.json({ message: 'Settings updated successfully', settings: db.settings });
});

// ==================== EXPORT & IMPORT & RESET ====================

app.get('/api/export', (req, res) => {
  const db = readDatabase();
  const format = req.query.format || 'json';
```

```
  if (format === 'csv') {
    // Generate CSV for attendance
    let csv = 'Subject Code,Subject Name,Classes Conducted,Classes Attended,Attendance Percentage,Status,Faculty\n';
    const threshold = db.settings.minAttendanceThreshold || 75;
    const margin = db.settings.warningMargin || 5;

    db.attendance.forEach(a => {
      const en = enrichAttendanceRecord(a, threshold, margin);
      csv += `"${en.code}","${en.name}",${en.conducted},${en.attended},${en.percentage !== null ? en.percentage + '%' : 'N/A'},"${en.status}","${en.faculty || ''}"\n`;
    });

    res.setHeader('Content-Type', 'text/csv');
    res.setHeader('Content-Disposition', 'attachment; filename="campus360_attendance.csv"');
    return res.send(csv);
  }

  res.setHeader('Content-Type', 'application/json');
  res.setHeader('Content-Disposition', 'attachment; filename="campus360_backup.json"');
  res.json({
    exportedAt: new Date().toISOString(),
    version: '1.0.0',
    app: 'Campus360 Attendance & GPA Module',
    data: db
  });
});

app.post('/api/import', (req, res) => {
  const { data } = req.body;
  if (!data || typeof data !== 'object') {
    return res.status(400).json({ error: 'Invalid import payload structure.' });
  }

  const db = {
    settings: data.settings || { minAttendanceThreshold: 75, warningMargin: 5, institutionRule: 'weighted_credits', gradeScale: [] },
    attendance: Array.isArray(data.attendance) ? data.attendance : [],
    syllabus: Array.isArray(data.syllabus) ? data.syllabus : [],
    gpaCourses: Array.isArray(data.gpaCourses) ? data.gpaCourses : [],
    cgpaSemesters: Array.isArray(data.cgpaSemesters) ? data.cgpaSemesters : []
  };

  writeDatabase(db);
  res.json({ message: 'Data imported and restored successfully.' });
});

app.post('/api/reset', (req, res) => {
  // Reset back to initial baseline demo data
  const defaultData = {
    settings: {
      minAttendanceThreshold: 75,
      warningMargin: 5,
      institutionRule: "weighted_credits",
      gradeScale: [
        { grade: "O", points: 10, description: "Outstanding (90-100%)" },
        { grade: "A+", points: 9, description: "Excellent (80-89%)" },
        { grade: "A", points: 8, description: "Very Good (70-79%)" },
        { grade: "B+", points: 7, description: "Good (60-69%)" },
        { grade: "B", points: 6, description: "Above Average (55-59%)" },
        { grade: "C", points: 5, description: "Average (50-54%)" },
        { grade: "P", points: 4, description: "Pass (40-49%)" },
        { grade: "F", points: 0, description: "Fail (<40%)" }
      ]
    },
    attendance: [
      { id: "att_1", code: "CS301", name: "Mathematics III", conducted: 40, attended: 34, faculty: "Dr. Sarah Jenkins", lastUpdated: "2026-10-25" },
      { id: "att_2", code: "CS302", name: "Physics & Engineering Electromagnetics", conducted: 36, attended: 25, faculty: "Prof. Alan Turing", lastUpdated: "2026-10-26" },
      { id: "att_3", code: "CS303", name: "Programming in C++", conducted: 32, attended: 28, faculty: "Dr. Grace Hopper", lastUpdated: "2026-10-27" },
      { id: "att_4", code: "CS304", name: "Data Structures & Algorithms", conducted: 28, attended: 22, faculty: "Prof. Donald Knuth", lastUpdated: "2026-10-26" },
      { id: "att_5", code: "CS305", name: "Digital Logic Design", conducted: 30, attended: 21, faculty: "Dr. Claude Shannon", lastUpdated: "2026-10-24" }
    ],
    syllabus: [
      {
        id: "syl_1", subjectCode: "CS301", subjectName: "Mathematics III", totalUnits: 5,
        units: [
          { unitNo: 1, title: "Linear Algebra & Vector Spaces", topicsTotal: 6, topicsCompleted: 6, status: "Completed" },
          { unitNo: 2, title: "Calculus & Multivariable Functions", topicsTotal: 8, topicsCompleted: 8, status: "Completed" },
          { unitNo: 3, title: "Differential Equations & Laplace Transforms", topicsTotal: 7, topicsCompleted: 4, status: "In Progress" },
          { unitNo: 4, title: "Complex Analysis & Mapping", topicsTotal: 6, topicsCompleted: 0, status:
```

```
"Not Started" },
```

```
          { unitNo: 5, title: "Numerical Methods & Iterations", topicsTotal: 5, topicsCompleted: 0, status: "Not Started" }
        ]
      },
      {
        id: "syl_2", subjectCode: "CS302", subjectName: "Physics & Engineering Electromagnetics", totalUnits: 4,
        units: [
          { unitNo: 1, title: "Electromagnetic Fields & Maxwell's Equations", topicsTotal: 6, topicsCompleted: 6, status: "Completed" },
          { unitNo: 2, title: "Optics & Wave Propagation", topicsTotal: 6, topicsCompleted: 3, status: "In Progress" },
          { unitNo: 3, title: "Quantum Mechanics Fundamentals", topicsTotal: 5, topicsCompleted: 0, status: "Not Started" },
          { unitNo: 4, title: "Thermodynamics & Statistical Physics", topicsTotal: 5, topicsCompleted: 0, status: "Not Started" }
        ]
      },
      {
        id: "syl_3", subjectCode: "CS303", subjectName: "Programming in C/C++", totalUnits: 5,
        units: [
          { unitNo: 1, title: "Language Basics, Flow & Functions", topicsTotal: 5, topicsCompleted: 5, status: "Completed" },
          { unitNo: 2, title: "Pointers, References & Memory Management", topicsTotal: 6, topicsCompleted: 6, status: "Completed" },
          { unitNo: 3, title: "Object-Oriented Design & Classes", topicsTotal: 7, topicsCompleted: 7, status: "Completed" },
          { unitNo: 4, title: "File Handling & Stream I/O", topicsTotal: 4, topicsCompleted: 2, status: "In Progress" },
          { unitNo: 5, title: "Templates, Exception Handling & STL", topicsTotal: 6, topicsCompleted: 1, status: "In Progress" }
        ]
      },
      {
        id: "syl_4", subjectCode: "CS304", subjectName: "Data Structures & Algorithms", totalUnits: 5,
        units: [
          { unitNo: 1, title: "Arrays, Stacks & Complexity Analysis", topicsTotal: 5, topicsCompleted: 5, status: "Completed" },
          { unitNo: 2, title: "Queues, Deques & Linked Lists", topicsTotal: 6, topicsCompleted: 6, status: "Completed" },
          { unitNo: 3, title: "Non-linear Structures: Trees & BSTs", topicsTotal: 7, topicsCompleted: 2, status: "In Progress" },
          { unitNo: 4, title: "Graphs & Minimum Spanning Trees", topicsTotal: 6, topicsCompleted: 0, status: "Not Started" },
          { unitNo: 5, title: "Sorting, Searching & Hash Tables", topicsTotal: 6, topicsCompleted: 0, status: "Not Started" }
        ]
      },
      {
        id: "syl_5", subjectCode: "CS305", subjectName: "Digital Logic Design", totalUnits: 4,
        units: [
          { unitNo: 1, title: "Boolean Algebra & Logic Gates", topicsTotal: 5, topicsCompleted: 5, status: "Completed" },
          { unitNo: 2, title: "Combinational Logic & Decoders/Multiplexers", topicsTotal: 6, topicsCompleted: 6, status: "Completed" },
          { unitNo: 3, title: "Sequential Logic: Flip-Flops & Counters", topicsTotal: 6, topicsCompleted: 1, status: "In Progress" },
          { unitNo: 4, title: "Memory Units & Programmable Logic (PLDs)", topicsTotal: 5, topicsCompleted: 0, status: "Not Started" }
        ]
      }
    ],
    gpaCourses: [
      { id: "gpa_1", code: "CS301", name: "Mathematics III", credits: 4, grade: "A", points: 8 },
      { id: "gpa_2", code: "CS302", name: "Physics & Engineering Electromagnetics", credits: 4, grade: "B+", points: 7 },
      { id: "gpa_3", code: "CS303", name: "Programming in C/C++", credits: 3, grade: "O", points: 10 },
      { id: "gpa_4", code: "CS304", name: "Data Structures & Algorithms", credits: 4, grade: "A+", points: 9 },
      { id: "gpa_5", code: "CS305", name: "Digital Logic Design", credits: 3, grade: "B", points: 6 },
      { id: "gpa_6", code: "CS306", name: "Data Structures Laboratory", credits: 2, grade: "O", points: 10 }
    ],
    cgpaSemesters: [
      { id: "sem_1", semester: "Semester 1", credits: 20, sgpa: 8.40 },
      { id: "sem_2", semester: "Semester 2", credits: 22, sgpa: 8.65 },
      { id: "sem_3", semester: "Semester 3", credits: 21, sgpa: 8.10 },
      { id: "sem_4", semester: "Semester 4 (Current)", credits: 20, sgpa: 8.25 }
    ]
  };

  writeDatabase(defaultData);
  res.json({ message: 'Database reset to initial default state.' });
});

// Fallback to index.html for client-side routing
app.get('*', (req, res) => {
  res.sendFile(path.join(__dirname, 'public', 'index.html'));
});

app.listen(PORT, () => {
  console.log(`================================================`);
  console.log(`Campus360 Attendance & GPA Module Backend Started`);
  console.log(`Server URL: http://localhost:${PORT}`);
  console.log(`REST API available at: http://localhost:${PORT}/api/`);
  console.log(`================================================`);
});
```

2. package.json

Project Manifest & Scripts (26 lines)

```
{
  "name": "campus360-attendance-tracker",
  "version": "1.0.0",
  "description": "Campus360 Attendance, Syllabus & GPA/CGPA Tracking Platform with REST API and Interactive Dashboard",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "node server.js"
  },
  "keywords": [
    "attendance",
    "gpa",
    "cgpa",
    "syllabus-tracker",
    "student-dashboard",
    "campus360"
  ],
  "author": "Campus360 Development Team",
  "license": "MIT",
  "dependencies": {
    "cors": "^2.8.5",
    "express": "^4.19.2",
    "pdfkit": "^0.20.2"
  }
}

```

3. public/index.html

Frontend Dashboard & SPA View (HTML5) (1044 lines)

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Campus360 - Attendance, Syllabus & GPA/CGPA Tracker</title>
  <link rel="stylesheet" href="css/style.css">
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link
    href="https://fonts.googleapis.com/css2?
family=Plus+Jakarta+Sans:wght@400;500;600;700;800&family=JetBrains+Mono:wght@400;500&display=swap"
    rel="stylesheet">
</head>

<body>

  <!-- Top Navigation Header -->
  <header class="navbar">
    <div class="nav-container">
      <div class="brand">
        <div class="brand-logo">
          <svg width="24" height="24" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2.2" stroke-linecap="round" stroke-linejoin="round">
            <path d="M22 10v6M2 10l10-5 10 5-10 5z" />
            <path d="M6 12v5c3 3 9 3 12 0v-5" />
          </svg>
        </div>
        <div class="brand-text">
          <span class="brand-title">Campus<span class="accent-text">360</span></span>
          <span class="brand-subtitle">Academic Suite</span>
        </div>
      </div>

      <nav class="nav-menu" id="navMenu">
        <button class="nav-link active" data-target="dashboard">
          <svg class="nav-icon" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
            <rect x="3" y="3" width="7" height="9" />
            <rect x="14" y="3" width="7" height="5" />
            <rect x="14" y="12" width="7" height="9" />
            <rect x="3" y="16" width="7" height="5" />
          </svg>
          Dashboard
        </button>
        <button class="nav-link" data-target="attendance">
          <svg class="nav-icon" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
            <path d="M9 11l3 3L22 4" />
            <path d="M21 12v7a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2h11" />
          </svg>
          Attendance
        </button>
        <button class="nav-link" data-target="syllabus">
          <svg class="nav-icon" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
            <path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20" />
            <path d="M6.5 2H20v20H6.5A2.5 2.5 0 0 1 4 19.5v-15A2.5 2.5 0 0 1 6.5 2z" />
          </svg>
          Syllabus
        </button>
        <button class="nav-link" data-target="gpa">
          <svg class="nav-icon" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
            <path d="M4 4h16c1.1 0 2 .9 2 2v12c0 1.1-.9 2-2 2H4c-1.1 0-2-.9-2-2V6c0-1.1.9-2 2-2z" />
            <line x1="12" y1="4" x2="12" y2="20" />
          </svg>
          Semester GPA
        </button>
        <button class="nav-link" data-target="cgpa">
          <svg class="nav-icon" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
            <polyline points="22 12 18 12 15 21 9 3 6 12 2 12" />
          </svg>
          Cumulative CGPA
        </button>
        <button class="nav-link" data-target="settings">
          <svg class="nav-icon" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
```

```
            <circle cx="12" cy="12" r="3" />
            <path
              d="M19.4 15a1.65 1.65 0 0 0 .33 1.82l.06.06a2 2 0 0 1 0 2.83 2 2 0 0 1-2.83 0l-.06-.06a1.65
1.65 0 0 0-1.82-.33 1.65 1.65 0 0 0-1 1.51V21a2 2 0 0 1-2 2 2 2 0 0 1-2-2v-.09A1.65 1.65 0 0 0 9 19.4a1.65
1.65 0 0 0-1.82.33l-.06.06a2 2 0 0 1-2.83 0 2 2 0 0 1 0-2.83l.06-.06a1.65 1.65 0 0 0 .33-1.82 1.65 1.65 0 0
0-1.51-1H3a2 2 0 0 1-2-2 2 2 0 0 1 2-2h.09A1.65 1.65 0 0 0 4.6 9a1.65 1.65 0 0 0-.33-1.82l-.06-.06a2 2 0 0
1 0-2.83 2 2 0 0 1 2.83 0l.06.06a1.65 1.65 0 0 0 1.82.33H9a1.65 1.65 0 0 0 1-1.51V3a2 2 0 0 1 2-2 2 2 0 0 1
2 2v.09a1.65 1.65 0 0 0 1 1.51 1.65 1.65 0 0 0 1.82-.33l.06-.06a2 2 0 0 1 2.83 0 2 2 0 0 1 0
2.83l-.06.06a1.65 1.65 0 0 0-.33 1.82V9a1.65 1.65 0 0 0 1.51 1H21a2 2 0 0 1 2 2 2 2 0 0 1-2 2h-.09a1.65
1.65 0 0 0-1.51 1z" />
          </svg>
          Settings
        </button>
      </div>
    </div>
  </header>

  <!-- Main Content Container -->
  <main class="main-content">

    <!-- ALERT BANNER FOR SYSTEM-WIDE ATTENDANCE / SYLLABUS WARNINGS -->
    <div id="systemAlertBanner" class="alert-banner hidden">
      <div class="alert-content">
        <div class="alert-icon" id="alertIcon">& þ </div>
        <div class="alert-text">
          <strong id="alertTitle">Attention Required:</strong>
          <span id="alertMessage">You have subjects with attendance below the mandatory threshold.</span>
        </div>
      </div>
      <button class="alert-close"
        onclick="document.getElementById('systemAlertBanner').classList.add('hidden')">&times;</button>
    </div>

    <!-- 1. DASHBOARD SECTION -->
    <section id="dashboard" class="content-section active">
      <div class="section-header">
        <div>
          <h1 class="page-title">Academic Overview</h1>
          <p class="page-subtitle">Real-time attendance metrics, syllabus completion status, and GPA/CGPA
insights</p>
        </div>
        <div class="header-actions">
          <button class="btn btn-secondary" onclick="exportData('json')">
            <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2">
              <path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4" />
              <polyline points="7 10 12 15 17 10" />
              <line x1="12" y1="15" x2="12" y2="3" />
            </svg>
            Export Backup
          </button>
          <button class="btn btn-primary" onclick="switchTab('attendance')">
            <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2">
              <line x1="12" y1="5" x2="12" y2="19" />
              <line x1="5" y1="12" x2="19" y2="12" />
            </svg>
            Update Attendance
          </button>
        </div>
      </div>

      <!-- KPI Metrics Grid -->
      <div class="kpi-grid">
        <!-- Overall Attendance Card -->
        <div class="kpi-card" id="kpiAttendanceCard">
          <div class="kpi-header">
            <span class="kpi-label">Overall Attendance</span>
            <span class="kpi-badge" id="kpiAttendanceBadge">Safe</span>
          </div>
          <div class="kpi-body">
```

```
            <div class="kpi-val-group">
              <span class="kpi-value" id="kpiOverallAttendance">--%</span>
              <span class="kpi-detail" id="kpiAttendanceRatio">0 / 0 classes attended</span>
            </div>
            <div class="progress-bar-wrapper">
              <div class="progress-bar-track">
                <div class="progress-bar-fill" id="kpiAttendanceProgress" style="width: 0%"></div>
              </div>
              <div class="progress-meta">
                <span id="kpiThresholdNote">Mandatory Threshold: 75%</span>
                <span id="kpiSafeBunksNote">--</span>
              </div>
            </div>
          </div>
        </div>

        <!-- Syllabus Completion Card -->
        <div class="kpi-card" id="kpiSyllabusCard">
          <div class="kpi-header">
            <span class="kpi-label">Syllabus Completion</span>
            <span class="kpi-badge info" id="kpiSyllabusBadge">On Track</span>
          </div>
          <div class="kpi-body">
            <div class="kpi-val-group">
              <span class="kpi-value" id="kpiOverallSyllabus">--%</span>
              <span class="kpi-detail" id="kpiSyllabusRatio">0 / 0 topics completed</span>
            </div>
            <div class="progress-bar-wrapper">
              <div class="progress-bar-track">
                <div class="progress-bar-fill progress-blue" id="kpiSyllabusProgress" style="width: 0%"></
div>
              </div>
              <div class="progress-meta">
                <span>Across All Subjects</span>
                <span id="kpiSyllabusStatusText">In Progress</span>
              </div>
            </div>
          </div>
        </div>

        <!-- Current Semester GPA Card -->
        <div class="kpi-card">
          <div class="kpi-header">
            <span class="kpi-label">Semester GPA</span>
            <span class="kpi-badge purple" id="kpiGpaBadge">Semester 4</span>
          </div>
          <div class="kpi-body">
            <div class="kpi-val-group">
              <span class="kpi-value accent-purple" id="kpiSemesterGPA">--</span>
              <span class="kpi-detail" id="kpiGpaCredits">0 registered credits</span>
            </div>
            <p class="kpi-subtext">Weighted SGPA: :2„7&VF—G2 r  ö–çG2’ ò £(Credits)</p>
          </div>
        </div>

        <!-- Cumulative CGPA Card -->
        <div class="kpi-card">
          <div class="kpi-header">
            <span class="kpi-label">Cumulative CGPA</span>
            <span class="kpi-badge teal" id="kpiCgpaBadge">Cumulative</span>
          </div>
          <div class="kpi-body">
            <div class="kpi-val-group">
              <span class="kpi-value accent-teal" id="kpiCumulativeCGPA">--</span>
              <span class="kpi-detail" id="kpiCgpaCredits">0 total earned credits</span>
            </div>
            <p class="kpi-subtext">Weighted CGPA across all completed semesters</p>
          </div>
        </div>
      </div>

      <!-- Quick Summary Cards & Visual Overview -->
      <div class="dashboard-grid-2col">
        <!-- Subject Attendance Summary Table -->
        <div class="card">
          <div class="card-header">
            <div>
              <h2 class="card-title">Subject Attendance Status</h2>
              <p class="card-subtitle">Current status & required classes to achieve minimum threshold</p>
```

```
            </div>
            <button class="btn btn-sm btn-outline" onclick="switchTab('attendance')">View All</button>
          </div>
          <div class="table-responsive">
            <table class="data-table">
              <thead>
                <tr>
                  <th>Course</th>
                  <th>Attended / Held</th>
                  <th>Percentage</th>
                  <th>Status</th>
                  <th>Action Needed</th>
                </tr>
              </thead>
              <tbody id="dashboardAttendanceList">
                <tr>
                  <td colspan="5" class="text-center py-4">Loading data...</td>
                </tr>
              </tbody>
            </table>
          </div>
        </div>

        <!-- Syllabus Progress per Subject -->
        <div class="card">
          <div class="card-header">
            <div>
              <h2 class="card-title">Syllabus Completion by Subject</h2>
              <p class="card-subtitle">Modules & topics coverage for upcoming examinations</p>
            </div>
            <button class="btn btn-sm btn-outline" onclick="switchTab('syllabus')">View Units</button>
          </div>
          <div class="syllabus-summary-list" id="dashboardSyllabusList">
            <p class="text-center py-4 text-muted">Loading syllabus status...</p>
          </div>
        </div>
      </div>

      <!-- Quick Action Shortcuts -->
      <div class="quick-shortcuts-panel">
        <h3 class="quick-title">Quick Actions</h3>
        <div class="shortcut-buttons">
          <button class="shortcut-btn" onclick="openAddSubjectModal()">
            <span class="shortcut-icon">'•</span>
            <span class="shortcut-label">Add Course Attendance</span>
          </button>
          <button class="shortcut-btn" onclick="openBulkImportModal()">
            <span class="shortcut-icon">Ø=ÜÄ</span>
            <span class="shortcut-label">Bulk CSV Import</span>
          </button>
          <button class="shortcut-btn" onclick="switchTab('gpa')">
            <span class="shortcut-icon">Ø>Ýî</span>
            <span class="shortcut-label">Calculate Semester GPA</span>
          </button>
          <button class="shortcut-btn" onclick="openTargetCgpaModal()">
            <span class="shortcut-icon">Ø<ß¯</span>
            <span class="shortcut-label">Target CGPA Planner</span>
          </button>
        </div>
      </div>
    </section>

    <!-- 2. ATTENDANCE TRACKER SECTION -->
    <section id="attendance" class="content-section">
      <div class="section-header">
        <div>
          <h1 class="page-title">Attendance Tracker</h1>
          <p class="page-subtitle">Track conducted and attended classes, calculate safe bunks and
attendance shortage recoveries</p>
        </div>
        <div class="header-actions">
          <button class="btn btn-outline" onclick="openBulkImportModal()">
            <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2">
              <path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4" />
              <polyline points="17 8 12 3 7 8" />
              <line x1="12" y1="3" x2="12" y2="15" />
            </svg>
            Bulk CSV Import
```

```
          </button>
          <button class="btn btn-primary" onclick="openAddSubjectModal()">
            <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2">    <line x1="12" y1="5" x2="12" y2="19" />
              <line x1="5" y1="12" x2="19" y2="12" />
            </svg>
            Add Subject
          </button>
        </div>
      </div>

      <!-- Quick Filter Bar -->
      <div class="filter-bar">
        <div class="search-box">
          <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2">  <circle cx="11" cy="11" r="8" />
            <line x1="21" y1="21" x2="16.65" y2="16.65" />
          </svg>
          <input type="text" id="attendanceSearchInput" placeholder="Search subject code, name, or
faculty..." onkeyup="filterAttendanceTable()">
        </div>
        <div class="filter-pills">
          <button class="filter-pill active" onclick="setAttendanceFilter('all', this)">All Courses</button>
          <button class="filter-pill pill-danger" onclick="setAttendanceFilter('danger', this)">Shortage
            (&lt;75%)</button>
          <button class="filter-pill pill-warning" onclick="setAttendanceFilter('warning', this)">Margin
            (70-75%)</button>
          <button class="filter-pill pill-safe" onclick="setAttendanceFilter('safe', this)">Safe (&ge;75%)</
button> </div>
      </div>

      <!-- Attendance Table Card -->
      <div class="card">
        <div class="table-responsive">
          <table class="data-table" id="attendanceTable">
            <thead>
              <tr>
                <th>Code & Name</th>
                <th>Conducted</th>
                <th>Attended</th>
                <th>Percentage</th>
                <th>Status</th>
                <th>Threshold Advice</th>
                <th>Quick Log</th>
                <th>Actions</th>
              </tr>
            </thead>
            <tbody id="attendanceTableBody">
              <!-- Rendered via attendance.js -->
            </tbody>
          </table>
        </div>
        <div class="table-footer-info">
          <span>Ø=Ü¡ <strong>Attendance Rule:</strong> Percentage = (Attended &divide; Conducted) &times; 100.
Minimum     threshold is configured in <a href="javascript:void(0)"
onclick="switchTab('settings')">Settings</a>.</span>
        </div>
      </div>

      <!-- Formula & Calculation Assumptions Box -->
      <div class="info-card">
        <h4 class="info-title">
          <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2">  <circle cx="12" cy="12" r="10" />
            <line x1="12" y1="16" x2="12" y2="12" />
            <line x1="12" y1="8" x2="12.01" y2="8" />
          </svg>
          Calculation Principles & Assumptions
        </h4>
        <div class="info-grid">
          <div class="info-item">
            <strong>Attendance Shortage Formula:</strong>
            <p>If attendance &lt; threshold, the smallest number of future consecutive classes $n$ you must
attend        without missing is calculated as:
              <code>n = # (Threshold × Conducted - 100 × Attended) / (100 - Threshold)# </code>
            </p>
          </div>
          <div class="info-item">
            <strong>Safe Bunk Formula:</strong>
```

```
            <p>If safely above threshold, the number of classes $m$ you can safely miss without dropping
below threshold is:
              <code>m = # (100 × Attended - Threshold × Conducted) / Threshold# </code>
            </p>
          </div>
          <div class="info-item full-width">
            <strong>Key Assumption:</strong> All upcoming classes carry equal credit weight. College rules
apply       strictly on session counts without rounding up unless explicitly permitted by university
statutes. </div>
        </div>
      </div>
    </section>

    <!-- 3. SYLLABUS COMPLETION TRACKER SECTION -->
    <section id="syllabus" class="content-section">
      <div class="section-header">
        <div>
          <h1 class="page-title">Syllabus Completion Tracker</h1>
          <p class="page-subtitle">Track unit-by-unit syllabus progress, completed topics, and exam
readiness for each course</p>
        </div>
        <div class="header-actions">
          <button class="btn btn-primary" onclick="openAddSyllabusSubjectModal()">
            <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2">    <line x1="12" y1="5" x2="12" y2="19" />
              <line x1="5" y1="12" x2="19" y2="12" />
            </svg>
            Add Subject Syllabus
          </button>
        </div>
      </div>

      <!-- Overall Syllabus Progress Banner -->
      <div class="syllabus-hero-card">
        <div class="hero-content">
          <div class="hero-stat">
            <span class="hero-pct" id="syllabusHeroPct">0%</span>
            <span class="hero-desc">Average Syllabus Finished</span>
          </div>
          <div class="hero-progress-container">
            <div class="hero-progress-bar">
              <div class="hero-progress-fill" id="syllabusHeroFill" style="width: 0%"></div>
            </div>
            <div class="hero-stats-row">
              <span id="syllabusHeroTopics">0 / 0 Topics Covered</span>
              <span id="syllabusHeroStatus">Overall Status: In Progress</span>
            </div>
          </div>
        </div>
      </div>

      <!-- Syllabus Subject Cards Grid -->
      <div class="syllabus-grid" id="syllabusCardsContainer">
        <!-- Rendered via syllabus.js -->
      </div>
    </section>

    <!-- 4. SEMESTER GPA CALCULATOR SECTION -->
    <section id="gpa" class="content-section">
      <div class="section-header">
        <div>
          <h1 class="page-title">Semester GPA Calculator</h1>
          <p class="page-subtitle">Calculate Semester Grade Point Average (SGPA) based on course credits
and letter   grades</p>
        </div>
        <div class="header-actions">
          <button class="btn btn-outline" onclick="resetGpaForm()">Reset Rows</button>
          <button class="btn btn-secondary" onclick="addGpaRow()">
            <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2">    <line x1="12" y1="5" x2="12" y2="19" />
              <line x1="5" y1="12" x2="19" y2="12" />
            </svg>
            Add Course Row
          </button>
          <button class="btn btn-primary" onclick="saveGpaToBackend()">Save Semester GPA</button>
        </div>
      </div>

```

```
      <div class="calculator-layout">
        <div class="calc-table-card card">
          <div class="table-responsive">
            <table class="data-table" id="gpaTable">
              <thead>
                <tr>
                  <th>Course Code</th>
                  <th>Course Name</th>
                  <th>Credits</th>
                  <th>Grade</th>
                  <th>Grade Point</th>
                  <th>Credit Points</th>
                  <th>Remove</th>
                </tr>
              </thead>
              <tbody id="gpaTableBody">
                <!-- Dynamically populated via gpa.js -->
              </tbody>
            </table>
          </div>
          <div class="table-action-footer">
            <button class="btn btn-sm btn-outline" onclick="addGpaRow()">+ Add Another Subject</button>
            <span class="text-muted text-sm">Grading scale conforms to configured Settings.</span>
          </div>
        </div>

        <!-- GPA Calculation Result & Formula Display -->
        <div class="calc-summary-card card">
          <h3 class="summary-title">GPA Computation</h3>
          <div class="gpa-score-display">
            <span class="gpa-number" id="calculatedGpa">0.00</span>
            <span class="gpa-scale">/ 10.00</span>
          </div>
          <div class="gpa-breakdown">
            <div class="breakdown-row">
              <span>Total Semester Credits:</span>
              <strong id="gpaTotalCredits">0</strong>
            </div>
            <div class="breakdown-row">
              <span>Total Grade Points (:2 2 r u “£Â÷7  ãà
              <strong id="gpaTotalPoints">0.00</strong>
            </div>
          </div>
          <div class="formula-box">
            <span class="formula-label">Official Formula:</span>
            <code>SGPA = :2 „6÷W'6R 7&VF—G2 r w& FR  ö–çG2’ ò £ (Course Credits)</code>
          </div>
          <button class="btn btn-primary w-full mt-4" onclick="transferGpaToCgpa()">
            Push SGPA to CGPA Calculator &rarr;
          </button>
        </div>
      </div>
    </section>

    <!-- 5. CUMULATIVE CGPA CALCULATOR SECTION -->
    <section id="cgpa" class="content-section">
      <div class="section-header">
        <div>
          <h1 class="page-title">Cumulative CGPA Calculator</h1>
          <p class="page-subtitle">Track multi-semester academic performance, credit accumulations, and
graduation  forecasts</p>
        </div>
        <div class="header-actions">
          <button class="btn btn-outline" onclick="openTargetCgpaModal()">
            <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2">    <circle cx="12" cy="12" r="10" />
              <circle cx="12" cy="12" r="6" />
              <circle cx="12" cy="12" r="2" />
            </svg>
            Target CGPA Planner
          </button>
          <button class="btn btn-secondary" onclick="addCgpaSemesterRow()">
            <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2">    <line x1="12" y1="5" x2="12" y2="19" />
              <line x1="5" y1="12" x2="19" y2="12" />
            </svg>
            Add Semester
          </button>
```

```
          <button class="btn btn-primary" onclick="saveCgpaToBackend()">Save CGPA</button>
        </div>
      </div>

      <div class="calculator-layout">
        <div class="calc-table-card card">
          <div class="table-responsive">
            <table class="data-table" id="cgpaTable">
              <thead>
                <tr>
                  <th>Semester</th>
                  <th>Registered Credits</th>
                  <th>Semester GPA (SGPA)</th>
                  <th>Credit Points Weighted</th>
                  <th>Remove</th>
                </tr>
              </thead>
              <tbody id="cgpaTableBody">
                <!-- Dynamically populated via cgpa.js -->
              </tbody>
            </table>
          </div>
          <div class="table-action-footer">
            <button class="btn btn-sm btn-outline" onclick="addCgpaSemesterRow()">+ Add Semester</button>
            <span class="text-muted text-sm">Weights each semester by registered credits according to
university    regulations.</span>
          </div>
        </div>

        <!-- CGPA Summary Card -->
        <div class="calc-summary-card card">
          <h3 class="summary-title">Cumulative Result</h3>
          <div class="gpa-score-display">
            <span class="gpa-number accent-teal" id="calculatedCgpa">0.00</span>
            <span class="gpa-scale">/ 10.00</span>
          </div>
          <div class="gpa-breakdown">
            <div class="breakdown-row">
              <span>Total Earned Credits:</span>
              <strong id="cgpaTotalCredits">0</strong>
            </div>
            <div class="breakdown-row">
              <span>Total Weighted SGPA Points:</span>
              <strong id="cgpaTotalPoints">0.00</strong>
            </div>
            <div class="breakdown-row">
              <span>Equivalent Percentage:</span>
              <strong id="cgpaPercentage">0.00%</strong>
            </div>
          </div>
          <div class="formula-box">
            <span class="formula-label">Weighted Formula:</span>
            <code>CGPA = :2 …6VÖW7FW" 7&VF—G2 r 4u  ’ ò £ (Semester Credits)</code>
          </div>
          <div class="cgpa-classification" id="cgpaClassification">
            <span>First Class with Distinction</span>
          </div>
        </div>
      </div>
    </section>

    <!-- 6. SETTINGS SECTION -->
    <section id="settings" class="content-section">
      <div class="section-header">
        <div>
          <h1 class="page-title">Module Settings & Configuration</h1>
          <p class="page-subtitle">Configure institutional attendance rules, threshold percentages, grading
scale, and  data backups</p>
        </div>
        <div class="header-actions">
          <button class="btn btn-primary" onclick="saveSettings()">Save Configuration</button>
        </div>
      </div>

      <div class="settings-grid">
        <!-- Attendance Threshold Configuration -->
        <div class="card">
          <h2 class="card-title">Attendance Thresholds & Rules</h2>
```

```
          <p class="card-subtitle">Set minimum required percentages and early warning margins</p>

          <div class="form-group mt-4">
            <label class="form-label" for="settingMinThreshold">
              Minimum Required Attendance Threshold (%)
            </label>
            <div class="range-with-input">
              <input type="range" id="settingThresholdRange" min="50" max="100" value="75"
                oninput="syncThresholdRange(this.value)">
              <div class="input-with-suffix">
                <input type="number" id="settingMinThreshold" min="50" max="100" value="75"
                  oninput="syncThresholdInput(this.value)">
                <span class="suffix">%</span>
              </div>
            </div>
            <span class="form-hint">Standard university threshold is typically 75%. Below this triggers
mandatory      shortage notices.</span>
          </div>

          <div class="form-group mt-4">
            <label class="form-label" for="settingWarningMargin">
              Warning Margin / Approach Buffer (%)
            </label>
            <div class="input-with-suffix">
              <input type="number" id="settingWarningMargin" min="1" max="20" value="5">
              <span class="suffix">%</span>
            </div>
            <span class="form-hint">Subjects within this percentage above the threshold (e.g. 75% to 80%)
will be        highlighted in Yellow.</span>
          </div>

          <div class="form-group mt-4">
            <label class="form-label">Calculation Rule:</label>
            <select class="form-select" id="settingInstitutionRule">
              <option value="weighted_credits" selected>Standard Weighted Credits & Class Counts</option>
              <option value="strict_session">Strict Session Counts (No Grace Attendance)</option>
              <option value="medical_concession">Medical Concession Threshold Allowed (65%)</option>
            </select>
          </div>
        </div>

        <!-- Grading Scale Mapping -->
        <div class="card">
          <div class="card-header-inline">
            <div>
              <h2 class="card-title">Institutional Grading Scale</h2>
              <p class="card-subtitle">Grade letters and their corresponding grade point equivalents</p>
            </div>
            <button class="btn btn-sm btn-outline" onclick="addGradeScaleRow()">+ Add Grade</button>
          </div>

          <div class="table-responsive mt-3">
            <table class="data-table" id="gradeScaleTable">
              <thead>
                <tr>
                  <th>Letter Grade</th>
                  <th>Grade Points</th>
                  <th>Description / Range</th>
                  <th>Action</th>
                </tr>
              </thead>
              <tbody id="gradeScaleTableBody">
                <!-- Dynamically populated from settings.js -->
              </tbody>
            </table>
          </div>
        </div>

        <!-- Data Backup, Export & Restore -->
        <div class="card full-width">
          <h2 class="card-title">Data Backup, Export & Disaster Recovery</h2>
          <p class="card-subtitle">Export your records to file or restore an existing backup snapshot</p>

          <div class="backup-actions-row">
            <div class="backup-card">
              <div class="backup-icon">Ø=Üå</div>
              <div class="backup-info">
                <h4>Full JSON Backup</h4>
```

```
                <p>Download complete snapshot including attendance, syllabus units, GPA courses, and custom
settings.       </p>
              </div>
              <button class="btn btn-outline" onclick="exportData('json')">Export JSON</button>
            </div>

            <div class="backup-card">
              <div class="backup-icon">Ø=ÜÊ</div>
              <div class="backup-info">
                <h4>Attendance CSV Report</h4>
                <p>Export subject-wise attendance spreadsheet for records or spreadsheet software.</p>
              </div>
              <button class="btn btn-outline" onclick="exportData('csv')">Export CSV</button>
            </div>

            <div class="backup-card">
              <div class="backup-icon">Ø=Üä</div>
              <div class="backup-info">
                <h4>Restore / Import Data</h4>
                <p>Upload a JSON backup file to restore all student data with confirmation.</p>
              </div>
              <label class="btn btn-secondary cursor-pointer">
                Upload Backup
                <input type="file" id="importFileInput" accept=".json" style="display:none;"
                  onchange="importDataFile(event)">
              </label>
            </div>

            <div class="backup-card danger-card">
              <div class="backup-icon">& þ </div>
              <div class="backup-info">
                <h4>Reset to Demo Data</h4>
                <p>Clear all custom modifications and restore standard campus sample records.</p>
              </div>
              <button class="btn btn-danger" onclick="confirmResetDatabase()">Reset Database</button>
            </div>
          </div>
        </div>
      </section>

  </main>

  <!-- ==================== MODALS ==================== -->

  <!-- Add/Edit Attendance Subject Modal -->
  <div class="modal-overlay hidden" id="subjectModal">
    <div class="modal-card">
      <div class="modal-header">
        <h3 class="modal-title" id="subjectModalTitle">Add Course Attendance</h3>
        <button class="modal-close" onclick="closeSubjectModal()">&times;</button>
      </div>
      <form id="subjectForm" onsubmit="handleSubjectFormSubmit(event)">
        <input type="hidden" id="modalSubjectId">
        <div class="modal-body">
          <div class="form-row">
            <div class="form-group flex-1">
              <label class="form-label" for="modalSubjectCode">Course Code *</label>
              <input type="text" id="modalSubjectCode" class="form-input" placeholder="e.g. CS301" required>
            </div>
            <div class="form-group flex-2">
              <label class="form-label" for="modalSubjectName">Course Name *</label>
              <input type="text" id="modalSubjectName" class="form-input" placeholder="e.g. Mathematics
III" required>
            </div>
          </div>

          <div class="form-row">
            <div class="form-group flex-1">
              <label class="form-label" for="modalClassesConducted">Total Classes Conducted *</label>
              <input type="number" id="modalClassesConducted" class="form-input" min="0" placeholder="e.g.
40" required     oninput="validateModalAttendance()">
            </div>
            <div class="form-group flex-1">
              <label class="form-label" for="modalClassesAttended">Classes Attended *</label>
              <input type="number" id="modalClassesAttended" class="form-input" min="0" placeholder="e.g.
34" required     oninput="validateModalAttendance()">
            </div>
          </div>
```

```
          <div class="form-group">
            <label class="form-label" for="modalFaculty">Faculty / Professor (Optional)</label>
            <input type="text" id="modalFaculty" class="form-input" placeholder="e.g. Dr. Sarah Jenkins">
          </div>

          <!-- Realtime Preview inside Modal -->
          <div class="modal-preview-box" id="modalAttendancePreview">
            <div class="preview-metric">
              <span>Estimated Attendance:</span>
              <strong id="modalPreviewPct">--%</strong>
            </div>
            <div class="preview-status">
              <span class="status-pill pill-safe" id="modalPreviewStatus">Valid</span>
            </div>
          </div>
          <div class="error-msg-box hidden" id="modalAttendanceError"></div>
        </div>

        <div class="modal-footer">
          <button type="button" class="btn btn-outline" onclick="closeSubjectModal()">Cancel</button>
          <button type="submit" class="btn btn-primary" id="saveSubjectBtn">Save Subject</button>
        </div>
      </form>
    </div>
  </div>

  <!-- Bulk CSV Import Modal -->
  <div class="modal-overlay hidden" id="bulkImportModal">
    <div class="modal-card modal-lg">
      <div class="modal-header">
        <h3 class="modal-title">Bulk CSV Attendance Importer</h3>
        <button class="modal-close" onclick="closeBulkImportModal()">&times;</button>
      </div>
      <div class="modal-body">
        <p class="text-sm text-muted mb-3">
          Upload an official attendance spreadsheet or CSV report. You can match the columns detected in
your CSV to the system's fields before importing.
        </p>

        <div class="file-drop-area" id="fileDropArea">
          <div class="drop-icon">Ø=ÜÁ</div>
          <p>Drag and drop your attendance CSV file here, or <label class="file-browse-link">browse<input
type="file"     id="csvFileInput" accept=".csv" style="display:none;"
onchange="handleCsvFileSelected(event)"></label>
          </p>
          <span class="text-xs text-muted">Supports comma, semicolon, or tab-delimited files</span>
        </div>

        <!-- Column Mapping Section (Visible after CSV loaded) -->
        <div id="columnMappingSection" class="hidden mt-4">
          <h4 class="subheading">Match CSV Columns</h4>
          <div class="mapping-grid">
            <div class="form-group">
              <label class="form-label">Course Name Column *</label>
              <select id="mapSubjectName" class="form-select"></select>
            </div>
            <div class="form-group">
              <label class="form-label">Course Code Column</label>
              <select id="mapSubjectCode" class="form-select"></select>
            </div>
            <div class="form-group">
              <label class="form-label">Classes Conducted Column *</label>
              <select id="mapConducted" class="form-select"></select>
            </div>
            <div class="form-group">
              <label class="form-label">Classes Attended Column *</label>
              <select id="mapAttended" class="form-select"></select>
            </div>
          </div>

          <div class="import-options-row mt-3">
            <label class="checkbox-label">
              <input type="checkbox" id="replaceExistingCheckbox">
              <span>Replace existing subjects instead of merging</span>
            </label>
            <button class="btn btn-sm btn-secondary" onclick="previewMappedCsv()">Preview Data</button>
          </div>

```

```
          <!-- Preview Table -->
          <div id="csvPreviewContainer" class="hidden mt-3">
            <h4 class="subheading">Detected Records Preview</h4>
            <div class="table-responsive max-h-56">
              <table class="data-table" id="csvPreviewTable">
                <thead>
                  <tr>
                    <th>#</th>
                    <th>Code</th>
                    <th>Course Name</th>
                    <th>Conducted</th>
                    <th>Attended</th>
                    <th>Attendance %</th>
                    <th>Status</th>
                  </tr>
                </thead>
                <tbody id="csvPreviewTableBody"></tbody>
              </table>
            </div>
          </div>
        </div>

        <div class="error-msg-box hidden mt-3" id="bulkImportError"></div>
      </div>

      <div class="modal-footer">
        <button type="button" class="btn btn-outline" onclick="closeBulkImportModal()">Cancel</button>
        <button type="button" class="btn btn-primary" id="confirmImportBtn" disabled
          onclick="executeBulkImport()">Confirm & Import Records</button>
      </div>
    </div>
  </div>

  <!-- Add/Edit Syllabus Unit Modal -->
  <div class="modal-overlay hidden" id="syllabusUnitModal">
    <div class="modal-card">
      <div class="modal-header">
        <h3 class="modal-title" id="syllabusModalTitle">Manage Syllabus Unit</h3>
        <button class="modal-close" onclick="closeSyllabusModal()">&times;</button>
      </div>
      <form id="syllabusUnitForm" onsubmit="handleSyllabusUnitSubmit(event)">
        <input type="hidden" id="sylSubjectId">
        <input type="hidden" id="sylUnitNo">
        <div class="modal-body">
          <div class="form-group">
            <label class="form-label" for="sylUnitTitle">Unit Title / Topic *</label>
            <input type="text" id="sylUnitTitle" class="form-input" placeholder="e.g. Unit 3: Differential
Equations"    required>
          </div>
          <div class="form-row">
            <div class="form-group flex-1">
              <label class="form-label" for="sylTopicsTotal">Total Sub-topics / Hours</label>
              <input type="number" id="sylTopicsTotal" class="form-input" min="1" value="5" required>
            </div>
            <div class="form-group flex-1">
              <label class="form-label" for="sylTopicsCompleted">Completed Sub-topics</label>
              <input type="number" id="sylTopicsCompleted" class="form-input" min="0" value="0" required>
            </div>
          </div>
          <div class="form-group">
            <label class="form-label" for="sylUnitStatus">Status</label>
            <select id="sylUnitStatus" class="form-select">
              <option value="Not Started">Not Started</option>
              <option value="In Progress">In Progress</option>
              <option value="Completed">Completed</option>
            </select>
          </div>
        </div>
        <div class="modal-footer">
          <button type="button" class="btn btn-outline" onclick="closeSyllabusModal()">Cancel</button>
          <button type="submit" class="btn btn-primary">Save Unit</button>
        </div>
      </form>
    </div>
  </div>

  <!-- Target CGPA Planner Modal -->
  <div class="modal-overlay hidden" id="targetCgpaModal">
```

```
    <div class="modal-card">
      <div class="modal-header">
        <h3 class="modal-title">Target CGPA Planner</h3>
        <button class="modal-close" onclick="closeTargetCgpaModal()">&times;</button>
      </div>
      <div class="modal-body">
        <p class="text-sm text-muted mb-3">
          Forecast what SGPA you need to achieve in remaining future semesters to reach your target graduation CGPA.</p>
        <div class="form-group">
          <label class="form-label">Desired Target Graduation CGPA</label>
          <input type="number" id="plannerTargetCgpa" class="form-input" step="0.01" min="1" max="10" value="8.50" oninput="calculateTargetPlanner()">
        </div>
        <div class="form-row">
          <div class="form-group flex-1">
            <label class="form-label">Remaining Semesters</label>
            <input type="number" id="plannerFutureSemesters" class="form-input" min="1" max="6" value="4"
              oninput="calculateTargetPlanner()">
          </div>
          <div class="form-group flex-1">
            <label class="form-label">Avg Credits per Semester</label>
            <input type="number" id="plannerAvgCredits" class="form-input" min="10" max="35" value="20"
              oninput="calculateTargetPlanner()">
          </div>
        </div>

        <div class="planner-result-box" id="plannerResultBox">
          <span class="planner-title">Required SGPA Each Remaining Semester:</span>
          <span class="planner-value" id="plannerRequiredSgpa">--</span>
          <p class="planner-note" id="plannerNote">Achievable with steady study schedule!</p>
        </div>
      </div>
      <div class="modal-footer">
        <button type="button" class="btn btn-primary" onclick="closeTargetCgpaModal()">Close Planner</button>
      </div>
    </div>
  </div>

  <!-- Formula & Documentation Modal -->
  <div class="modal-overlay hidden" id="infoModal">
    <div class="modal-card modal-lg">
      <div class="modal-header">
        <h3 class="modal-title">Campus360 Formula Guide & Reference</h3>
        <button class="modal-close" onclick="closeInfoModal()">&times;</button>
      </div>
      <div class="modal-body">
        <div class="doc-section">
          <h4>1. Attendance Calculations</h4>
          <p><strong>Attendance Percentage:</strong> <code>(Attended Classes ÷ Conducted Classes) × 100</code></p> <p><strong>Classes Needed to Reach Threshold:</strong> If attendance is below mandatory threshold $T$ (e.g. 75%), the minimum subsequent classes $n$ that must be attended consecutively is:</p>
          <code>n = # (T × Conducted - 100 × Attended) / (100 - T)# </code>
          <p><strong>Safe Bunk Allowance:</strong> If safely above threshold, the number of classes $m$ you can safely skip without falling below $T$ is:</p>
          <code>m = # (100 × Attended - T × Conducted) / T# </code>
        </div>

        <div class="doc-section mt-3">
          <h4>2. Semester GPA (SGPA)</h4>
          <p>Calculated as credit-weighted average of grade points obtained in all enrolled courses:</p>
          <code>SGPA = :2 „6÷W'6R 7&VF—G2 r w& FR  ö–çG2’ ò £ (Course Credits)</code>
        </div>

        <div class="doc-section mt-3">
          <h4>3. Cumulative Grade Point Average (CGPA)</h4>
          <p>Calculated across all completed semesters weighted by earned semester credits:</p>
          <code>CGPA = :2 …6VÖW7FW" 7&VF—G2 r 4u  ’ ò £ (Semester Credits)</code>
        </div>

        <div class="doc-section mt-3">
          <h4>4. Syllabus Completion Rate</h4>
          <p>Tracks unit and topic milestones against total syllabus chapters:</p>
          <code>Subject Completion % = (Completed Topics ÷ Total Topics) × 100</code>
        </div>
      </div>
      <div class="modal-footer">
        <button type="button" class="btn btn-primary" onclick="closeInfoModal()">Got it</button>
```

```
      </div>
    </div>
  </div>

  <!-- Confirmation Dialog Modal -->
  <div class="modal-overlay hidden" id="confirmModal">
    <div class="modal-card modal-sm">
      <div class="modal-header">
        <h3 class="modal-title" id="confirmModalTitle">Please Confirm</h3>
        <button class="modal-close" onclick="closeConfirmModal(false)">&times;</button>
      </div>
      <div class="modal-body">
        <p id="confirmModalMessage">Are you sure you want to proceed with this action?</p>
      </div>
      <div class="modal-footer">
        <button type="button" class="btn btn-outline" onclick="closeConfirmModal(false)">Cancel</button>
        <button type="button" class="btn btn-danger" id="confirmModalBtn"
          onclick="closeConfirmModal(true)">Confirm</button>
      </div>
    </div>
  </div>

  <!-- Toast Notification Container -->
  <div class="toast-container" id="toastContainer"></div>

  <!-- Scripts -->
  <script src="js/app.js"></script>
  <script src="js/attendance.js"></script>
  <script src="js/syllabus.js"></script>
  <script src="js/gpa.js"></script>
  <script src="js/cgpa.js"></script>
  <script src="js/settings.js"></script>
</body>

</html>
```

4. public/css/style.css

Complete UI Design System & Responsive Styles (CSS3) (1560 lines)

```css
/* Campus360 - Clean Modern Design System */
:root {
  --primary: #4f46e5;
  --primary-hover: #4338ca;
  --primary-light: #eef2ff;
  --primary-glow: rgba(79, 70, 229, 0.15);

  --secondary: #0ea5e9;
  --secondary-hover: #0284c7;

  --success: #10b981;
  --success-bg: #ecfdf5;
  --success-border: #a7f3d0;

  --warning: #f59e0b;
  --warning-bg: #fffbeb;
  --warning-border: #fde68a;

  --danger: #ef4444;
  --danger-bg: #fef2f2;
  --danger-border: #fecaca;

  --bg-main: #f8fafc;
  --bg-card: #ffffff;
  --bg-subtle: #f1f5f9;

  --text-primary: #0f172a;
  --text-secondary: #475569;
  --text-muted: #64748b;
  --text-light: #94a3b8;

  --border: #e2e8f0;
  --border-focus: #818cf8;

  --radius-sm: 6px;
  --radius-md: 10px;
  --radius-lg: 16px;
  --radius-full: 9999px;

  --shadow-sm: 0 1px 2px 0 rgb(0 0 0 / 0.05);
  --shadow-md: 0 4px 6px -1px rgb(0 0 0 / 0.07), 0 2px 4px -2px rgb(0 0 0 / 0.05);
  --shadow-lg: 0 10px 15px -3px rgb(0 0 0 / 0.08), 0 4px 6px -4px rgb(0 0 0 / 0.03);
  --shadow-xl: 0 20px 25px -5px rgb(0 0 0 / 0.1), 0 8px 10px -6px rgb(0 0 0 / 0.05);

  --font-sans: 'Plus Jakarta Sans', system-ui, -apple-system, sans-serif;
  --font-mono: 'JetBrains Mono', monospace;
  --transition: all 0.2s cubic-bezier(0.4, 0, 0.2, 1);
}

/* Base Reset */
*, *::before, *::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: var(--font-sans);
  background-color: var(--bg-main);
  color: var(--text-primary);
  line-height: 1.5;
  -webkit-font-smoothing: antialiased;
  min-height: 100vh;
  display: flex;
  flex-direction: column;
}

/* Navbar */
.navbar {
  background: #ffffff;
  border-bottom: 1px solid var(--border);
  position: sticky;
  top: 0;
```

```
  z-index: 100;
  box-shadow: var(--shadow-sm);
}

.nav-container {
  max-width: 1360px;
  margin: 0 auto;
  padding: 0 1.5rem;
  height: 68px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1.5rem;
}

.brand {
  display: flex;
  align-items: center;
  gap: 0.75rem;
  text-decoration: none;
  cursor: pointer;
}

.brand-logo {
  width: 40px;
  height: 40px;
  background: linear-gradient(135deg, #4f46e5 0%, #3b82f6 100%);
  border-radius: var(--radius-md);
  display: flex;
  align-items: center;
  justify-content: center;
  color: #fff;
  box-shadow: 0 4px 10px rgba(79, 70, 229, 0.3);
}

.brand-text {
  display: flex;
  flex-direction: column;
}

.brand-title {
  font-size: 1.25rem;
  font-weight: 800;
  letter-spacing: -0.02em;
  color: #0f172a;
}

.accent-text {
  color: var(--primary);
}

.brand-subtitle {
  font-size: 0.7rem;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 0.08em;
  color: var(--text-muted);
}

.nav-menu {
  display: flex;
  align-items: center;
  gap: 0.35rem;
}

.nav-link {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  padding: 0.55rem 0.9rem;
  border-radius: var(--radius-md);
  background: transparent;
  border: none;
  font-size: 0.88rem;
  font-weight: 600;
  color: var(--text-secondary);
  cursor: pointer;
  transition: var(--transition);
```

```
}

.nav-icon {
  width: 17px;
  height: 17px;
}

.nav-link:hover {
  background: var(--bg-subtle);
  color: var(--text-primary);
}

.nav-link.active {
  background: var(--primary-light);
  color: var(--primary);
}

.nav-actions {
  display: flex;
  align-items: center;
  gap: 0.85rem;
}

.status-indicator {
  display: flex;
  align-items: center;
  gap: 0.4rem;
  padding: 0.35rem 0.75rem;
  background: var(--bg-subtle);
  border: 1px solid var(--border);
  border-radius: var(--radius-full);
  font-size: 0.75rem;
  font-weight: 600;
  color: var(--text-secondary);
}

.status-dot {
  width: 8px;
  height: 8px;
  border-radius: 50%;
  background: #cbd5e1;
}

.status-indicator.online .status-dot {
  background: var(--success);
  box-shadow: 0 0 8px rgba(16, 185, 129, 0.6);
}

.icon-btn {
  width: 38px;
  height: 38px;
  display: flex;
  align-items: center;
  justify-content: center;
  border: 1px solid var(--border);
  border-radius: var(--radius-md);
  background: #fff;
  color: var(--text-secondary);
  cursor: pointer;
  transition: var(--transition);
}

.icon-btn:hover {
  background: var(--bg-subtle);
  color: var(--text-primary);
}

.icon-btn svg {
  width: 18px;
  height: 18px;
}

.mobile-toggle-btn {
  display: none;
  flex-direction: column;
  gap: 5px;
  width: 36px;
  height: 36px;
```

```
  background: none;
  border: none;
  cursor: pointer;
  padding: 4px;
}

.mobile-toggle-btn span {
  display: block;
  height: 2px;
  width: 100%;
  background: var(--text-primary);
  border-radius: 2px;
}

/* Main Content Area */
.main-content {
  flex: 1;
  max-width: 1360px;
  width: 100%;
  margin: 0 auto;
  padding: 2rem 1.5rem 4rem;
}

.content-section {
  display: none;
  animation: fadeIn 0.25s ease-out;
}

.content-section.active {
  display: block;
}

@keyframes fadeIn {
  from { opacity: 0; transform: translateY(6px); }
  to { opacity: 1; transform: translateY(0); }
}

/* Alert Banner */
.alert-banner {
  margin-bottom: 1.5rem;
  padding: 1rem 1.25rem;
  background: var(--danger-bg);
  border: 1px solid var(--danger-border);
  border-radius: var(--radius-md);
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1rem;
  box-shadow: var(--shadow-sm);
}

.alert-banner.hidden {
  display: none;
}

.alert-content {
  display: flex;
  align-items: center;
  gap: 0.75rem;
}

.alert-icon {
  font-size: 1.25rem;
}

.alert-text strong {
  color: #991b1b;
  font-weight: 700;
  margin-right: 0.35rem;
}

.alert-text span {
  color: #b91c1c;
  font-size: 0.92rem;
}

.alert-close {
  background: none;
```

```
  border: none;
  font-size: 1.4rem;
  color: #991b1b;
  cursor: pointer;
  line-height: 1;
}

/* Section Header */
.section-header {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  gap: 1rem;
  margin-bottom: 2rem;
  flex-wrap: wrap;
}

.page-title {
  font-size: 1.75rem;
  font-weight: 800;
  letter-spacing: -0.03em;
  color: var(--text-primary);
}

.page-subtitle {
  font-size: 0.92rem;
  color: var(--text-muted);
  margin-top: 0.25rem;
}

.header-actions {
  display: flex;
  align-items: center;
  gap: 0.75rem;
}

/* KPI Cards Grid */
.kpi-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));
  gap: 1.25rem;
  margin-bottom: 2rem;
}

.kpi-card {
  background: #ffffff;
  border: 1px solid var(--border);
  border-radius: var(--radius-lg);
  padding: 1.4rem;
  box-shadow: var(--shadow-sm);
  display: flex;
  flex-direction: column;
  justify-content: space-between;
  transition: var(--transition);
}

.kpi-card:hover {
  transform: translateY(-2px);
  box-shadow: var(--shadow-md);
  border-color: #cbd5e1;
}

.kpi-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-bottom: 0.75rem;
}

.kpi-label {
  font-size: 0.84rem;
  font-weight: 700;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  color: var(--text-muted);
}

.kpi-badge {
```

```
  font-size: 0.72rem;
  font-weight: 700;
  padding: 0.25rem 0.6rem;
  border-radius: var(--radius-full);
  background: var(--success-bg);
  color: #047857;
  border: 1px solid var(--success-border);
}

.kpi-badge.warning {
  background: var(--warning-bg);
  color: #b45309;
  border-color: var(--warning-border);
}

.kpi-badge.danger {
  background: var(--danger-bg);
  color: #b91c1c;
  border-color: var(--danger-border);
}

.kpi-badge.info {
  background: #e0f2fe;
  color: #0369a1;
  border-color: #bae6fd;
}

.kpi-badge.purple {
  background: #f3e8ff;
  color: #7e22ce;
  border-color: #e9d5ff;
}

.kpi-badge.teal {
  background: #ccfbf1;
  color: #0f766e;
  border-color: #99f6e4;
}

.kpi-val-group {
  margin-bottom: 0.65rem;
}

.kpi-value {
  font-size: 2.2rem;
  font-weight: 800;
  line-height: 1.1;
  color: var(--text-primary);
  display: block;
}

.kpi-value.accent-purple { color: #7c3aed; }
.kpi-value.accent-teal { color: #0d9488; }

.kpi-detail {
  font-size: 0.82rem;
  color: var(--text-muted);
  display: block;
  margin-top: 0.2rem;
}

.kpi-subtext {
  font-size: 0.78rem;
  color: var(--text-muted);
  border-top: 1px dashed var(--border);
  padding-top: 0.5rem;
  margin-top: 0.5rem;
}

/* Progress Bar Component */
.progress-bar-wrapper {
  margin-top: 0.6rem;
}

.progress-bar-track {
  height: 8px;
  background: var(--bg-subtle);
  border-radius: var(--radius-full);
```

```
  overflow: hidden;
}

.progress-bar-fill {
  height: 100%;
  background: var(--success);
  border-radius: var(--radius-full);
  transition: width 0.4s ease-out;
}

.progress-bar-fill.progress-yellow { background: var(--warning); }
.progress-bar-fill.progress-red { background: var(--danger); }
.progress-bar-fill.progress-blue { background: var(--secondary); }

.progress-meta {
  display: flex;
  justify-content: space-between;
  font-size: 0.75rem;
  color: var(--text-muted);
  margin-top: 0.4rem;
}

/* Cards & Layouts */
.card {
  background: #ffffff;
  border: 1px solid var(--border);
  border-radius: var(--radius-lg);
  padding: 1.5rem;
  box-shadow: var(--shadow-sm);
  margin-bottom: 1.5rem;
}

.card-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-bottom: 1.25rem;
  flex-wrap: wrap;
  gap: 0.75rem;
}

.card-header-inline {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1rem;
}

.card-title {
  font-size: 1.15rem;
  font-weight: 700;
  letter-spacing: -0.02em;
  color: var(--text-primary);
}

.card-subtitle {
  font-size: 0.85rem;
  color: var(--text-muted);
  margin-top: 0.2rem;
}

.dashboard-grid-2col {
  display: grid;
  grid-template-columns: 1.35fr 1fr;
  gap: 1.5rem;
  margin-bottom: 2rem;
}

/* Data Tables */
.table-responsive {
  width: 100%;
  overflow-x: auto;
}

.data-table {
  width: 100%;
  border-collapse: separate;
  border-spacing: 0;
```

```
  font-size: 0.88rem;
}

.data-table th {
  background: #f8fafc;
  color: var(--text-secondary);
  font-weight: 700;
  text-align: left;
  padding: 0.75rem 1rem;
  border-bottom: 1px solid var(--border);
  font-size: 0.8rem;
  text-transform: uppercase;
  letter-spacing: 0.04em;
  white-space: nowrap;
}

.data-table td {
  padding: 0.85rem 1rem;
  border-bottom: 1px solid var(--border);
  vertical-align: middle;
  color: var(--text-secondary);
}

.data-table tr:last-child td {
  border-bottom: none;
}

.data-table tbody tr {
  transition: background-color 0.15s ease;
}

.data-table tbody tr:hover {
  background-color: #f8fafc;
}

.table-footer-info {
  margin-top: 1rem;
  padding-top: 0.85rem;
  border-top: 1px solid var(--border);
  font-size: 0.82rem;
  color: var(--text-muted);
}

.table-action-footer {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-top: 1rem;
  padding-top: 1rem;
  border-top: 1px solid var(--border);
}

/* Status Pills */
.status-pill {
  display: inline-flex;
  align-items: center;
  gap: 0.35rem;
  padding: 0.25rem 0.65rem;
  border-radius: var(--radius-full);
  font-size: 0.75rem;
  font-weight: 700;
  white-space: nowrap;
}

.pill-safe {
  background: var(--success-bg);
  color: #047857;
  border: 1px solid var(--success-border);
}

.pill-warning {
  background: var(--warning-bg);
  color: #b45309;
  border: 1px solid var(--warning-border);
}

.pill-danger {
  background: var(--danger-bg);
```

```
  color: #b91c1c;
  border: 1px solid var(--danger-border);
}

.pill-neutral {
  background: var(--bg-subtle);
  color: var(--text-muted);
  border: 1px solid var(--border);
}

/* Quick Increment Buttons in Attendance Table */
.quick-log-group {
  display: inline-flex;
  gap: 0.35rem;
}

.quick-btn {
  padding: 0.3rem 0.6rem;
  font-size: 0.75rem;
  font-weight: 700;
  border-radius: var(--radius-sm);
  border: 1px solid var(--border);
  background: #fff;
  cursor: pointer;
  transition: var(--transition);
}

.quick-btn.present {
  color: #047857;
  border-color: var(--success-border);
}
.quick-btn.present:hover {
  background: var(--success-bg);
}

.quick-btn.absent {
  color: #b91c1c;
  border-color: var(--danger-border);
}
.quick-btn.absent:hover {
  background: var(--danger-bg);
}

/* Buttons */
.btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 0.5rem;
  padding: 0.55rem 1.15rem;
  font-size: 0.88rem;
  font-weight: 600;
  border-radius: var(--radius-md);
  cursor: pointer;
  border: 1px solid transparent;
  transition: var(--transition);
  text-decoration: none;
  white-space: nowrap;
}

.btn-primary {
  background: var(--primary);
  color: #ffffff;
}
.btn-primary:hover {
  background: var(--primary-hover);
  box-shadow: 0 4px 12px rgba(79, 70, 229, 0.25);
}

.btn-secondary {
  background: var(--secondary);
  color: #ffffff;
}
.btn-secondary:hover {
  background: var(--secondary-hover);
}

.btn-outline {
```

```
  background: #ffffff;
  border-color: var(--border);
  color: var(--text-secondary);
}
.btn-outline:hover {
  background: var(--bg-subtle);
  color: var(--text-primary);
  border-color: #cbd5e1;
}

.btn-danger {
  background: var(--danger);
  color: #ffffff;
}
.btn-danger:hover {
  background: #dc2626;
}

.btn-sm {
  padding: 0.35rem 0.75rem;
  font-size: 0.8rem;
  border-radius: var(--radius-sm);
}

.btn-icon {
  padding: 0.4rem;
  border-radius: var(--radius-sm);
  background: none;
  border: 1px solid transparent;
  color: var(--text-muted);
  cursor: pointer;
}
.btn-icon:hover {
  background: var(--bg-subtle);
  color: var(--text-primary);
  border-color: var(--border);
}

.w-full { width: 100%; }
.mt-3 { margin-top: 0.75rem; }
.mt-4 { margin-top: 1rem; }
.mb-3 { margin-bottom: 0.75rem; }
.cursor-pointer { cursor: pointer; }

/* Filter Bar */
.filter-bar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1rem;
  margin-bottom: 1.25rem;
  flex-wrap: wrap;
}

.search-box {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  background: #ffffff;
  border: 1px solid var(--border);
  border-radius: var(--radius-md);
  padding: 0.45rem 0.85rem;
  width: 320px;
  max-width: 100%;
}

.search-box input {
  border: none;
  background: none;
  outline: none;
  font-size: 0.88rem;
  width: 100%;
  color: var(--text-primary);
}

.filter-pills {
  display: flex;
  align-items: center;
```

```
  gap: 0.4rem;
  flex-wrap: wrap;
}

.filter-pill {
  padding: 0.35rem 0.85rem;
  border-radius: var(--radius-full);
  border: 1px solid var(--border);
  background: #ffffff;
  font-size: 0.8rem;
  font-weight: 600;
  color: var(--text-secondary);
  cursor: pointer;
  transition: var(--transition);
}

.filter-pill:hover {
  background: var(--bg-subtle);
}

.filter-pill.active {
  background: var(--primary);
  color: #ffffff;
  border-color: var(--primary);
}

.filter-pill.pill-danger.active {
  background: var(--danger);
  border-color: var(--danger);
  color: #fff;
}
.filter-pill.pill-warning.active {
  background: var(--warning);
  border-color: var(--warning);
  color: #fff;
}
.filter-pill.pill-safe.active {
  background: var(--success);
  border-color: var(--success);
  color: #fff;
}

/* Info Box */
.info-card {
  background: #f8fafc;
  border: 1px solid var(--border);
  border-radius: var(--radius-lg);
  padding: 1.25rem 1.5rem;
  margin-top: 1.5rem;
}

.info-title {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  font-size: 0.95rem;
  font-weight: 700;
  color: var(--text-primary);
  margin-bottom: 0.75rem;
}

.info-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 1.25rem;
  font-size: 0.85rem;
  color: var(--text-secondary);
}

.info-item.full-width {
  grid-column: span 2;
  border-top: 1px dashed var(--border);
  padding-top: 0.75rem;
}

.info-item strong {
  display: block;
  color: var(--text-primary);
```

```
  margin-bottom: 0.25rem;
}

.info-item code {
  font-family: var(--font-mono);
  background: #e2e8f0;
  padding: 0.15rem 0.4rem;
  border-radius: 4px;
  font-size: 0.8rem;
}

/* Quick Shortcuts Panel */
.quick-shortcuts-panel {
  margin-top: 1.5rem;
}

.quick-title {
  font-size: 1rem;
  font-weight: 700;
  color: var(--text-secondary);
  margin-bottom: 0.75rem;
}

.shortcut-buttons {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 1rem;
}

.shortcut-btn {
  display: flex;
  align-items: center;
  gap: 0.85rem;
  background: #ffffff;
  border: 1px solid var(--border);
  border-radius: var(--radius-md);
  padding: 0.9rem 1.1rem;
  cursor: pointer;
  transition: var(--transition);
  text-align: left;
}

.shortcut-btn:hover {
  border-color: var(--primary);
  transform: translateY(-2px);
  box-shadow: var(--shadow-sm);
}

.shortcut-icon {
  font-size: 1.4rem;
}

.shortcut-label {
  font-size: 0.88rem;
  font-weight: 600;
  color: var(--text-primary);
}

/* Syllabus Tracker Specifics */
.syllabus-hero-card {
  background: linear-gradient(135deg, #1e1b4b 0%, #312e81 100%);
  color: #ffffff;
  border-radius: var(--radius-lg);
  padding: 1.75rem;
  margin-bottom: 2rem;
  box-shadow: var(--shadow-md);
}

.hero-content {
  display: flex;
  align-items: center;
  gap: 2.5rem;
  flex-wrap: wrap;
}

.hero-stat {
  display: flex;
  flex-direction: column;
```

```
}

.hero-pct {
  font-size: 3rem;
  font-weight: 800;
  line-height: 1;
}

.hero-desc {
  font-size: 0.85rem;
  color: #c7d2fe;
  margin-top: 0.35rem;
}

.hero-progress-container {
  flex: 1;
  min-width: 250px;
}

.hero-progress-bar {
  height: 12px;
  background: rgba(255, 255, 255, 0.2);
  border-radius: var(--radius-full);
  overflow: hidden;
  margin-bottom: 0.6rem;
}

.hero-progress-fill {
  height: 100%;
  background: #38bdf8;
  border-radius: var(--radius-full);
  transition: width 0.4s ease;
}

.hero-stats-row {
  display: flex;
  justify-content: space-between;
  font-size: 0.85rem;
  color: #e0e7ff;
}

.syllabus-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(380px, 1fr));
  gap: 1.5rem;
}

.syllabus-card {
  background: #ffffff;
  border: 1px solid var(--border);
  border-radius: var(--radius-lg);
  padding: 1.5rem;
  box-shadow: var(--shadow-sm);
  display: flex;
  flex-direction: column;
  justify-content: space-between;
}

.syllabus-card-header {
  display: flex;
  justify-content: space-between;
  align-items: flex-start;
  margin-bottom: 1rem;
}

.syllabus-subject-code {
  font-size: 0.75rem;
  font-weight: 700;
  color: var(--primary);
  text-transform: uppercase;
}

.syllabus-subject-title {
  font-size: 1.1rem;
  font-weight: 700;
  color: var(--text-primary);
  margin-top: 0.15rem;
}
```

```css

.unit-list {
  display: flex;
  flex-direction: column;
  gap: 0.75rem;
  margin: 1.25rem 0;
}

.unit-item {
  background: #f8fafc;
  border: 1px solid var(--border);
  border-radius: var(--radius-md);
  padding: 0.75rem;
  display: flex;
  flex-direction: column;
  gap: 0.45rem;
}

.unit-item-top {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 0.5rem;
}

.unit-title-text {
  font-size: 0.85rem;
  font-weight: 600;
  color: var(--text-primary);
}

.unit-topic-controls {
  display: flex;
  align-items: center;
  gap: 0.35rem;
}

.unit-btn {
  width: 24px;
  height: 24px;
  border-radius: 4px;
  border: 1px solid var(--border);
  background: #fff;
  cursor: pointer;
  display: flex;
  align-items: center;
  justify-content: center;
  font-weight: bold;
}
.unit-btn:hover {
  background: var(--primary-light);
  color: var(--primary);
}

.unit-progress-track {
  height: 6px;
  background: #e2e8f0;
  border-radius: var(--radius-full);
  overflow: hidden;
}

.unit-progress-fill {
  height: 100%;
  background: var(--secondary);
  border-radius: var(--radius-full);
}

.unit-progress-fill.done {
  background: var(--success);
}

/* Calculator Layouts (GPA / CGPA) */
.calculator-layout {
  display: grid;
  grid-template-columns: 1.8fr 1fr;
  gap: 1.5rem;
  align-items: flex-start;
}
```

```
1088
1089 .calc-summary-card {
1090   position: sticky;
1091   top: 90px;
1092   background: #ffffff;
1093   border: 1px solid var(--border);
1094   border-radius: var(--radius-lg);
1095   padding: 1.75rem;
1096   box-shadow: var(--shadow-md);
1097 }
1098
1099 .summary-title {
1100   font-size: 1.05rem;
1101   font-weight: 700;
1102   color: var(--text-secondary);
1103   margin-bottom: 1rem;
1104 }
1105
1106 .gpa-score-display {
1107   display: flex;
1108   align-items: baseline;
1109   gap: 0.4rem;
1110   margin-bottom: 1.25rem;
1111 }
1112
1113 .gpa-number {
1114   font-size: 3.5rem;
1115   font-weight: 800;
1116   line-height: 1;
1117   color: var(--primary);
1118 }
1119
1120 .gpa-scale {
1121   font-size: 1.25rem;
1122   font-weight: 600;
1123   color: var(--text-muted);
1124 }
1125
1126 .gpa-breakdown {
1127   display: flex;
1128   flex-direction: column;
1129   gap: 0.5rem;
1130   padding-bottom: 1rem;
1131   border-bottom: 1px solid var(--border);
1132   margin-bottom: 1rem;
1133 }
1134
1135 .breakdown-row {
1136   display: flex;
1137   justify-content: space-between;
1138   font-size: 0.88rem;
1139   color: var(--text-secondary);
1140 }
1141
1142 .formula-box {
1143   background: #f8fafc;
1144   border: 1px dashed var(--border);
1145   border-radius: var(--radius-md);
1146   padding: 0.75rem;
1147 }
1148
1149 .formula-label {
1150   display: block;
1151   font-size: 0.75rem;
1152   font-weight: 700;
1153   text-transform: uppercase;
1154   color: var(--text-muted);
1155   margin-bottom: 0.25rem;
1156 }
1157
1158 .formula-box code {
1159   font-family: var(--font-mono);
1160   font-size: 0.8rem;
1161   color: var(--text-primary);
1162   display: block;
1163 }
1164
1165 .cgpa-classification {
```

```
  margin-top: 1rem;
  text-align: center;
  padding: 0.6rem;
  background: #f0fdf4;
  border: 1px solid #bbf7d0;
  border-radius: var(--radius-md);
  font-weight: 700;
  font-size: 0.85rem;
  color: #166534;
}

/* Settings Grid */
.settings-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 1.5rem;
}

.full-width {
  grid-column: span 2;
}

.range-with-input {
  display: flex;
  align-items: center;
  gap: 1.25rem;
}

.range-with-input input[type="range"] {
  flex: 1;
  accent-color: var(--primary);
  cursor: pointer;
}

.input-with-suffix {
  position: relative;
  width: 100px;
}

.input-with-suffix input {
  width: 100%;
  padding: 0.55rem 1.6rem 0.55rem 0.75rem;
  border: 1px solid var(--border);
  border-radius: var(--radius-md);
  font-weight: 600;
}

.input-with-suffix .suffix {
  position: absolute;
  right: 0.75rem;
  top: 50%;
  transform: translateY(-50%);
  color: var(--text-muted);
  font-weight: 600;
  pointer-events: none;
}

.form-group {
  display: flex;
  flex-direction: column;
  gap: 0.35rem;
  margin-bottom: 1rem;
}

.form-label {
  font-size: 0.85rem;
  font-weight: 600;
  color: var(--text-primary);
}

.form-input, .form-select {
  padding: 0.6rem 0.85rem;
  border: 1px solid var(--border);
  border-radius: var(--radius-md);
  font-family: var(--font-sans);
  font-size: 0.88rem;
  outline: none;
  transition: var(--transition);
```

```
  background: #ffffff;
  color: var(--text-primary);
}

.form-input:focus, .form-select:focus {
  border-color: var(--border-focus);
  box-shadow: 0 0 0 3px var(--primary-glow);
}

.form-hint {
  font-size: 0.78rem;
  color: var(--text-muted);
}

.form-row {
  display: flex;
  gap: 1rem;
}

.flex-1 { flex: 1; }
.flex-2 { flex: 2; }

/* Backup & Restore Cards */
.backup-actions-row {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: 1.25rem;
  margin-top: 1rem;
}

.backup-card {
  border: 1px solid var(--border);
  border-radius: var(--radius-md);
  padding: 1.25rem;
  background: #f8fafc;
  display: flex;
  flex-direction: column;
  justify-content: space-between;
  gap: 1rem;
}

.backup-card.danger-card {
  background: #fef2f2;
  border-color: #fee2e2;
}

.backup-icon {
  font-size: 1.75rem;
}

.backup-info h4 {
  font-size: 0.95rem;
  font-weight: 700;
  margin-bottom: 0.3rem;
}

.backup-info p {
  font-size: 0.8rem;
  color: var(--text-muted);
}

/* Modals */
.modal-overlay {
  position: fixed;
  inset: 0;
  background: rgba(15, 23, 42, 0.6);
  backdrop-filter: blur(4px);
  z-index: 1000;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 1.5rem;
  animation: fadeIn 0.2s ease-out;
}

.modal-overlay.hidden {
  display: none;
}
```

```
.modal-card {
  background: #ffffff;
  border-radius: var(--radius-lg);
  width: 100%;
  max-width: 520px;
  box-shadow: var(--shadow-xl);
  max-height: 90vh;
  display: flex;
  flex-direction: column;
  overflow: hidden;
}

.modal-card.modal-lg {
  max-width: 780px;
}

.modal-card.modal-sm {
  max-width: 420px;
}

.modal-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 1.25rem 1.5rem;
  border-bottom: 1px solid var(--border);
}

.modal-title {
  font-size: 1.15rem;
  font-weight: 700;
  color: var(--text-primary);
}

.modal-close {
  background: none;
  border: none;
  font-size: 1.5rem;
  color: var(--text-muted);
  cursor: pointer;
  line-height: 1;
}

.modal-body {
  padding: 1.5rem;
  overflow-y: auto;
}

.modal-footer {
  display: flex;
  justify-content: flex-end;
  gap: 0.75rem;
  padding: 1.1rem 1.5rem;
  border-top: 1px solid var(--border);
  background: #f8fafc;
}

/* Modal Specific UI */
.modal-preview-box {
  background: #f8fafc;
  border: 1px solid var(--border);
  border-radius: var(--radius-md);
  padding: 0.75rem 1rem;
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-top: 0.75rem;
}

.file-drop-area {
  border: 2px dashed #cbd5e1;
  border-radius: var(--radius-md);
  padding: 2rem;
  text-align: center;
  background: #f8fafc;
  cursor: pointer;
}
```

```
1400
1401 .drop-icon {
1402   font-size: 2.2rem;
1403   margin-bottom: 0.5rem;
1404 }
1405
1406 .file-browse-link {
1407   color: var(--primary);
1408   font-weight: 700;
1409   cursor: pointer;
1410   text-decoration: underline;
1411 }
1412
1413 .subheading {
1414   font-size: 0.95rem;
1415   font-weight: 700;
1416   margin-bottom: 0.65rem;
1417   color: var(--text-primary);
1418 }
1419
1420 .mapping-grid {
1421   display: grid;
1422   grid-template-columns: 1fr 1fr;
1423   gap: 1rem;
1424 }
1425
1426 .import-options-row {
1427   display: flex;
1428   align-items: center;
1429   justify-content: space-between;
1430   gap: 1rem;
1431 }
1432
1433 .checkbox-label {
1434   display: flex;
1435   align-items: center;
1436   gap: 0.5rem;
1437   font-size: 0.85rem;
1438   cursor: pointer;
1439 }
1440
1441 .max-h-56 {
1442   max-height: 14rem;
1443 }
1444
1445 .error-msg-box {
1446   background: var(--danger-bg);
1447   border: 1px solid var(--danger-border);
1448   color: #b91c1c;
1449   padding: 0.65rem 0.9rem;
1450   border-radius: var(--radius-md);
1451   font-size: 0.82rem;
1452 }
1453
1454 .planner-result-box {
1455   background: #ecfdf5;
1456   border: 1px solid #a7f3d0;
1457   border-radius: var(--radius-md);
1458   padding: 1.25rem;
1459   margin-top: 1rem;
1460   text-align: center;
1461 }
1462
1463 .planner-title {
1464   display: block;
1465   font-size: 0.85rem;
1466   font-weight: 600;
1467   color: #065f46;
1468 }
1469
1470 .planner-value {
1471   display: block;
1472   font-size: 2.5rem;
1473   font-weight: 800;
1474   color: #047857;
1475   line-height: 1.1;
1476   margin: 0.35rem 0;
1477 }
```

```
.planner-note {
  font-size: 0.8rem;
  color: #047857;
}

/* Toast Container */
.toast-container {
  position: fixed;
  bottom: 1.5rem;
  right: 1.5rem;
  z-index: 2000;
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
}

.toast {
  padding: 0.75rem 1.2rem;
  border-radius: var(--radius-md);
  background: #0f172a;
  color: #ffffff;
  font-size: 0.88rem;
  font-weight: 500;
  box-shadow: var(--shadow-lg);
  display: flex;
  align-items: center;
  gap: 0.6rem;
  animation: toastSlide 0.25s ease-out;
}

.toast.success { background: #065f46; border-left: 4px solid #10b981; }
.toast.error { background: #7f1d1d; border-left: 4px solid #ef4444; }
.toast.info { background: #1e3a8a; border-left: 4px solid #3b82f6; }

@keyframes toastSlide {
  from { transform: translateX(100%); opacity: 0; }
  to { transform: translateX(0); opacity: 1; }
}

/* Responsive Breakpoints */
@media (max-width: 1024px) {
  .dashboard-grid-2col,
  .calculator-layout,
  .settings-grid {
    grid-template-columns: 1fr;
  }
  .full-width {
    grid-column: span 1;
  }
}

@media (max-width: 768px) {
  .nav-menu {
    display: none;
    position: absolute;
    top: 68px;
    left: 0;
    right: 0;
    background: #ffffff;
    border-bottom: 1px solid var(--border);
    flex-direction: column;
    padding: 1rem;
    box-shadow: var(--shadow-lg);
  }
  .nav-menu.open {
    display: flex;
  }
  .mobile-toggle-btn {
    display: flex;
  }
  .kpi-grid {
    grid-template-columns: 1fr;
  }
  .info-grid {
    grid-template-columns: 1fr;
  }
  .info-item.full-width {
```

```
    grid-column: span 1;
  }
}
```

5. public/js/app.js

App Controller, Sync Engine & Modal Manager (JavaScript) (524 lines)

```
/**
 * Campus360 - Main Application Controller
 * Handles SPA navigation, API / LocalStorage persistence sync, KPI updates, and global modals.
 */

const API_BASE = '/api';

window.App = {
  isBackendConnected: false,
  settings: {
    minAttendanceThreshold: 75,
    warningMargin: 5,
    institutionRule: 'weighted_credits',
    gradeScale: []
  },
  attendance: [],
  syllabus: [],
  gpaCourses: [],
  cgpaSemesters: []
};

// ==================== INITIALIZATION ====================

document.addEventListener('DOMContentLoaded', async () => {
  setupNavigation();
  setupMobileMenu();
  setupDocumentationModal();

  // Load application data (try API first, fallback to LocalStorage)
  await loadAppData();

  // Initialize individual sub-modules
  if (window.initAttendanceModule) window.initAttendanceModule();
  if (window.initSyllabusModule) window.initSyllabusModule();
  if (window.initGpaModule) window.initGpaModule();
  if (window.initCgpaModule) window.initCgpaModule();
  if (window.initSettingsModule) window.initSettingsModule();

  // Render initial dashboard metrics
  renderDashboard();
});

// ==================== NAVIGATION ROUTER ====================

function setupNavigation() {
  const navLinks = document.querySelectorAll('.nav-link');
  navLinks.forEach(link => {
    link.addEventListener('click', () => {
      const target = link.getAttribute('data-target');
      switchTab(target);
    });
  });
}

function switchTab(tabId) {
  // Update nav links
  document.querySelectorAll('.nav-link').forEach(link => {
    if (link.getAttribute('data-target') === tabId) {
      link.classList.add('active');
    } else {
      link.classList.remove('active');
    }
  });

  // Switch content section
  document.querySelectorAll('.content-section').forEach(sec => {
    if (sec.id === tabId) {
      sec.classList.add('active');
    } else {
      sec.classList.remove('active');
    }
  });

```

```
  // Close mobile menu if opened
  const navMenu = document.getElementById('navMenu');
  if (navMenu) navMenu.classList.remove('open');

  // Specific tab refreshes
  if (tabId === 'dashboard') renderDashboard();
  if (tabId === 'attendance' && window.renderAttendanceTable) window.renderAttendanceTable();
  if (tabId === 'syllabus' && window.renderSyllabusCards) window.renderSyllabusCards();
  if (tabId === 'gpa' && window.renderGpaTable) window.renderGpaTable();
  if (tabId === 'cgpa' && window.renderCgpaTable) window.renderCgpaTable();
  if (tabId === 'settings' && window.renderSettingsForm) window.renderSettingsForm();

  window.scrollTo({ top: 0, behavior: 'smooth' });
}

function setupMobileMenu() {
  const mobileBtn = document.getElementById('mobileMenuBtn');
  const navMenu = document.getElementById('navMenu');
  if (mobileBtn && navMenu) {
    mobileBtn.addEventListener('click', () => {
      navMenu.classList.toggle('open');
    });
  }
}

// ==================== DATA SYNC (BACKEND API + LOCALSTORAGE) ====================

async function loadAppData() {
  const syncBadge = document.getElementById('syncStatusBadge');
  const syncText = document.getElementById('syncStatusText');

  try {
    const res = await fetch(`${API_BASE}/health`, { signal: AbortSignal.timeout(3000) });
    if (res.ok) {
      window.App.isBackendConnected = true;
      if (syncBadge && syncText) {
        syncBadge.className = 'status-indicator online';
        syncText.textContent = 'Server Connected';
      }
      // Load from server
      await fetchAllFromBackend();
      return;
    }
  } catch (err) {
    console.warn('Backend server not reachable, using Browser LocalStorage fallback:', err);
  }

  // LocalStorage Fallback
  window.App.isBackendConnected = false;
  if (syncBadge && syncText) {
    syncBadge.className = 'status-indicator';
    syncText.textContent = 'Offline (Local)';
  }
  loadFromLocalStorage();
}

async function fetchAllFromBackend() {
  try {
    const [settingsRes, attRes, sylRes, gpaRes, cgpaRes] = await Promise.all([
      fetch(`${API_BASE}/settings`).then(r => r.json()),
      fetch(`${API_BASE}/attendance`).then(r => r.json()),
      fetch(`${API_BASE}/syllabus`).then(r => r.json()),
      fetch(`${API_BASE}/gpa`).then(r => r.json()),
      fetch(`${API_BASE}/cgpa`).then(r => r.json())
    ]);

    window.App.settings = settingsRes;
    window.App.attendance = attRes;
    window.App.syllabus = sylRes;
    window.App.gpaCourses = gpaRes.courses || [];
    window.App.cgpaSemesters = cgpaRes.semesters || [];

    // Cache locally as secondary mirror
    saveToLocalStorage();
  } catch (err) {
    console.error('Failed to load from backend:', err);
    loadFromLocalStorage();
  }
```

```
}

function loadFromLocalStorage() {
  const saved = localStorage.getItem('campus360_data');
  if (saved) {
    try {
      const parsed = JSON.parse(saved);
      window.App.settings = parsed.settings || window.App.settings;
      window.App.attendance = parsed.attendance || [];
      window.App.syllabus = parsed.syllabus || [];
      window.App.gpaCourses = parsed.gpaCourses || [];
      window.App.cgpaSemesters = parsed.cgpaSemesters || [];
    } catch (e) {
      console.error('Error parsing local storage:', e);
    }
  }
}

function saveToLocalStorage() {
  const payload = {
    settings: window.App.settings,
    attendance: window.App.attendance,
    syllabus: window.App.syllabus,
    gpaCourses: window.App.gpaCourses,
    cgpaSemesters: window.App.cgpaSemesters
  };
  localStorage.setItem('campus360_data', JSON.stringify(payload));
}

// ==================== DASHBOARD RENDERING ====================

function renderDashboard() {
  const threshold = parseFloat(window.App.settings.minAttendanceThreshold) || 75;
  const margin = parseFloat(window.App.settings.warningMargin) || 5;

  // 1. Attendance Metrics
  let totalConducted = 0;
  let totalAttended = 0;
  let shortageCount = 0;
  let warningCount = 0;

  const attendanceList = window.App.attendance || [];
  attendanceList.forEach(item => {
    const cond = parseInt(item.conducted, 10) || 0;
    const att = parseInt(item.attended, 10) || 0;
    totalConducted += cond;
    totalAttended += att;

    if (cond > 0) {
      const pct = (att / cond) * 100;
      if (pct < threshold - margin) shortageCount++;
      else if (pct < threshold) warningCount++;
    }
  });

  const overallAttPct = totalConducted > 0
    ? Math.round((totalAttended / totalConducted) * 1000) / 10
    : 0;

  // KPI UI Updates
  const kpiOverallAtt = document.getElementById('kpiOverallAttendance');
  const kpiAttRatio = document.getElementById('kpiAttendanceRatio');
  const kpiAttProgress = document.getElementById('kpiAttendanceProgress');
  const kpiAttBadge = document.getElementById('kpiAttendanceBadge');
  const kpiSafeBunksNote = document.getElementById('kpiSafeBunksNote');
  const kpiThresholdNote = document.getElementById('kpiThresholdNote');

  if (kpiOverallAtt) kpiOverallAtt.textContent = totalConducted > 0 ? `${overallAttPct}%` : 'N/A';
  if (kpiAttRatio) kpiAttRatio.textContent = `${totalAttended} / ${totalConducted} classes attended`;
  if (kpiThresholdNote) kpiThresholdNote.textContent = `Mandatory Threshold: ${threshold}%`;

  if (kpiAttProgress) {
    kpiAttProgress.style.width = `${Math.min(100, overallAttPct)}%`;
    if (overallAttPct >= threshold) {
      kpiAttProgress.className = 'progress-bar-fill';
      if (kpiAttBadge) {
        kpiAttBadge.className = 'kpi-badge';
        kpiAttBadge.textContent = 'Safe';
```

```
      }
      const bunks = Math.floor((100 * totalAttended - threshold * totalConducted) / threshold);
      if (kpiSafeBunksNote) kpiSafeBunksNote.textContent = bunks > 0 ? `Can miss ${bunks} classes safely` : 'On boundary!';
    } else if (overallAttPct >= threshold - margin) {
      kpiAttProgress.className = 'progress-bar-fill progress-yellow';
      if (kpiAttBadge) {
        kpiAttBadge.className = 'kpi-badge warning';
        kpiAttBadge.textContent = 'Near Margin';
      }
      const needed = Math.ceil((threshold * totalConducted - 100 * totalAttended) / (100 - threshold));
      if (kpiSafeBunksNote) kpiSafeBunksNote.textContent = `Need ${needed} classes to reach ${threshold}%`;
    } else {
      kpiAttProgress.className = 'progress-bar-fill progress-red';
      if (kpiAttBadge) {
        kpiAttBadge.className = 'kpi-badge danger';
        kpiAttBadge.textContent = 'Shortage Alert';
      }
      const needed = Math.ceil((threshold * totalConducted - 100 * totalAttended) / (100 - threshold));
      if (kpiSafeBunksNote) kpiSafeBunksNote.textContent = `Must attend next ${needed} classes!`;
    }
  }

  // 2. Syllabus Completion Metrics
  let totalTopics = 0;
  let completedTopics = 0;
  (window.App.syllabus || []).forEach(sub => {
    (sub.units || []).forEach(u => {
      totalTopics += (parseInt(u.topicsTotal, 10) || 0);
      completedTopics += (parseInt(u.topicsCompleted, 10) || 0);
    });
  });

  const overallSyllabusPct = totalTopics > 0
    ? Math.round((completedTopics / totalTopics) * 100)
    : 0;

  const kpiOverallSyllabus = document.getElementById('kpiOverallSyllabus');
  const kpiSyllabusRatio = document.getElementById('kpiSyllabusRatio');
  const kpiSyllabusProgress = document.getElementById('kpiSyllabusProgress');
  const kpiSyllabusStatusText = document.getElementById('kpiSyllabusStatusText');

  if (kpiOverallSyllabus) kpiOverallSyllabus.textContent = `${overallSyllabusPct}%`;
  if (kpiSyllabusRatio) kpiSyllabusRatio.textContent = `${completedTopics} / ${totalTopics} topics completed`;
  if (kpiSyllabusProgress) kpiSyllabusProgress.style.width = `${overallSyllabusPct}%`;
  if (kpiSyllabusStatusText) {
    if (overallSyllabusPct >= 80) kpiSyllabusStatusText.textContent = 'Ahead of Schedule';
    else if (overallSyllabusPct >= 50) kpiSyllabusStatusText.textContent = 'On Track';
    else kpiSyllabusStatusText.textContent = 'Needs Focus';
  }

  // 3. GPA & CGPA Metrics
  let gpaCreds = 0, gpaPts = 0;
  (window.App.gpaCourses || []).forEach(c => {
    const cred = parseFloat(c.credits) || 0;
    const pt = parseFloat(c.points) || 0;
    gpaCreds += cred;
    gpaPts += (cred * pt);
  });
  const sgpa = gpaCreds > 0 ? (Math.round((gpaPts / gpaCreds) * 100) / 100).toFixed(2) : '0.00';
  const kpiGpa = document.getElementById('kpiSemesterGPA');
  const kpiGpaCredits = document.getElementById('kpiGpaCredits');
  if (kpiGpa) kpiGpa.textContent = sgpa;
  if (kpiGpaCredits) kpiGpaCredits.textContent = `${gpaCreds} registered credits`;

  let cgpaCreds = 0, cgpaPts = 0;
  (window.App.cgpaSemesters || []).forEach(s => {
    const cred = parseFloat(s.credits) || 0;
    const val = parseFloat(s.sgpa) || 0;
    cgpaCreds += cred;
    cgpaPts += (cred * val);
  });
  const cgpa = cgpaCreds > 0 ? (Math.round((cgpaPts / cgpaCreds) * 100) / 100).toFixed(2) : '0.00';
  const kpiCgpa = document.getElementById('kpiCumulativeCGPA');
  const kpiCgpaCredits = document.getElementById('kpiCgpaCredits');
  if (kpiCgpa) kpiCgpa.textContent = cgpa;
  if (kpiCgpaCredits) kpiCgpaCredits.textContent = `${cgpaCreds} total earned credits`;

  // 4. Alert Banner Display
```

```
  const alertBanner = document.getElementById('systemAlertBanner');
  const alertTitle = document.getElementById('alertTitle');
  const alertMsg = document.getElementById('alertMessage');

  if (alertBanner) {
    if (shortageCount > 0) {
      alertBanner.classList.remove('hidden');
      alertTitle.textContent = `Attendance Shortage Alert (${shortageCount} Subject${shortageCount > 1 ? 's' : ''})`;
      alertMsg.textContent = `You have subjects falling below ${threshold}%. Review required classes immediately to prevent exam debarment.`;
    } else if (warningCount > 0) {
      alertBanner.classList.remove('hidden');
      alertTitle.textContent = `Approaching Threshold Margin:`;
      alertMsg.textContent = `${warningCount} subject(s) are near the ${threshold}% margin. Maintain regular attendance.`;
    } else {
      alertBanner.classList.add('hidden');
    }
  }

  // 5. Render Dashboard Attendance Table Preview
  renderDashboardAttendanceTable();

  // 6. Render Dashboard Syllabus Progress Preview
  renderDashboardSyllabusPreview();
}

function renderDashboardAttendanceTable() {
  const tbody = document.getElementById('dashboardAttendanceList');
  if (!tbody) return;

  const threshold = parseFloat(window.App.settings.minAttendanceThreshold) || 75;
  const margin = parseFloat(window.App.settings.warningMargin) || 5;

  if (window.App.attendance.length === 0) {
    tbody.innerHTML = `<tr><td colspan="5" class="text-center py-4 text-muted">No attendance records yet. Click "Add Subject" to begin.</td></tr>`;
    return;
  }

  tbody.innerHTML = window.App.attendance.map(item => {
    const cond = parseInt(item.conducted, 10) || 0;
    const att = parseInt(item.attended, 10) || 0;

    let pctDisplay = 'Not available yet';
    let statusPill = '<span class="status-pill pill-neutral">N/A</span>';
    let advice = 'No classes conducted yet';

    if (cond > 0) {
      const pct = Math.round((att / cond) * 1000) / 10;
      pctDisplay = `${pct}%`;

      if (pct >= threshold) {
        statusPill = `<span class="status-pill pill-safe">Safe</span>`;
        const bunks = Math.floor((100 * att - threshold * cond) / threshold);
        advice = bunks > 0 ? `Can safely miss ${bunks} class${bunks > 1 ? 'es' : ''}` : 'On margin (attend all)';
      } else if (pct >= threshold - margin) {
        statusPill = `<span class="status-pill pill-warning">Near Margin</span>`;
        const needed = Math.ceil((threshold * cond - 100 * att) / (100 - threshold));
        advice = `Attend next ${needed} class${needed > 1 ? 'es' : ''}`;
      } else {
        statusPill = `<span class="status-pill pill-danger">Shortage</span>`;
        const needed = Math.ceil((threshold * cond - 100 * att) / (100 - threshold));
        advice = `<strong>Attend next ${needed} classes!</strong>`;
      }
    }

    return `
      <tr>
        <td>
          <div class="font-semibold text-primary">${item.name}</div>
          <span class="text-xs text-muted">${item.code || ''}</span>
        </td>
        <td><strong>${att}</strong> / ${cond}</td>
        <td><strong>${pctDisplay}</strong></td>
        <td>${statusPill}</td>
        <td class="text-sm">${advice}</td>
      </tr>
    `;
  }).join('');
}
```

```
386
387 function renderDashboardSyllabusPreview() {
388   const container = document.getElementById('dashboardSyllabusList');
389   if (!container) return;
390
391   if (window.App.syllabus.length === 0) {
392     container.innerHTML = `<p class="text-center py-4 text-muted">No syllabus tracked yet. Add subjects to
see completion progress.</p>`;
393     return;
394   }
395
396   container.innerHTML = window.App.syllabus.map(s => {
397     let tot = 0, comp = 0;
398     (s.units || []).forEach(u => {
399       tot += (parseInt(u.topicsTotal, 10) || 0);
400       comp += (parseInt(u.topicsCompleted, 10) || 0);
401     });
402     const pct = tot > 0 ? Math.round((comp / tot) * 100) : 0;
403
404     return `
405       <div class="syllabus-summary-item mb-3">
406         <div class="flex justify-between text-sm mb-1">
407           <span class="font-semibold">${s.subjectName} (${s.subjectCode || ''})</span>
408           <strong>${pct}% (${comp}/${tot} Topics)</strong>
409         </div>
410         <div class="progress-bar-track">
411           <div class="progress-bar-fill progress-blue" style="width: ${pct}%"></div>
412         </div>
413       </div>
414     `;
415   }).join('');
416 }
417
418 // ==================== TOAST NOTIFICATIONS ====================
419
420 function showToast(message, type = 'info') {
421   const container = document.getElementById('toastContainer');
422   if (!container) return;
423
424   const toast = document.createElement('div');
425   toast.className = `toast ${type}`;
426
427   let icon = '!9þ ';
428   if (type === 'success') icon = '' ';
429   if (type === 'error') icon = ''L';
430
431   toast.innerHTML = `<span>${icon}</span> <span>${message}</span>`;
432   container.appendChild(toast);
433
434   setTimeout(() => {
435     toast.style.opacity = '0';
436     toast.style.transform = 'translateY(10px)';
437     setTimeout(() => toast.remove(), 250);
438   }, 3500);
439 }
440
441 // ==================== MODAL SYSTEM ====================
442
443 function openConfirmModal(title, message, onConfirm) {
444   const modal = document.getElementById('confirmModal');
445   const titleEl = document.getElementById('confirmModalTitle');
446   const msgEl = document.getElementById('confirmModalMessage');
447
448   if (titleEl) titleEl.textContent = title;
449   if (msgEl) msgEl.textContent = message;
450
451   window._confirmCallback = onConfirm;
452   modal.classList.remove('hidden');
453 }
454
455 function closeConfirmModal(isConfirmed) {
456   const modal = document.getElementById('confirmModal');
457   modal.classList.add('hidden');
458   if (isConfirmed && typeof window._confirmCallback === 'function') {
459     window._confirmCallback();
460   }
461   window._confirmCallback = null;
462 }
463
```

```
function setupDocumentationModal() {
  const infoBtn = document.getElementById('infoBtn');
  const infoModal = document.getElementById('infoModal');
  if (infoBtn && infoModal) {
    infoBtn.addEventListener('click', () => infoModal.classList.remove('hidden'));
  }
}

function closeInfoModal() {
  const infoModal = document.getElementById('infoModal');
  if (infoModal) infoModal.classList.add('hidden');
}

// ==================== EXPORT DATA ====================

async function exportData(format = 'json') {
  if (window.App.isBackendConnected) {
    window.location.href = `${API_BASE}/export?format=${format}`;
    showToast(`Exported ${format.toUpperCase()} successfully!`, 'success');
  } else {
    // Generate file download directly in browser
    if (format === 'csv') {
      let csv = 'Subject Code,Subject Name,Classes Conducted,Classes Attended,Attendance
Percentage,Faculty\n';
      window.App.attendance.forEach(a => {
        const pct = a.conducted > 0 ? `${Math.round((a.attended / a.conducted) * 1000) / 10}%` : 'N/A';
        csv += `"${a.code || ''}","${a.name}",${a.conducted},${a.attended},${pct},"${a.faculty || ''}"\n`;
      });
      downloadBlob(csv, 'campus360_attendance.csv', 'text/csv');
      showToast('Attendance CSV downloaded!', 'success');
    } else {
      const payload = {
        exportedAt: new Date().toISOString(),
        version: '1.0.0',
        app: 'Campus360 Module',
        data: {
          settings: window.App.settings,
          attendance: window.App.attendance,
          syllabus: window.App.syllabus,
          gpaCourses: window.App.gpaCourses,
          cgpaSemesters: window.App.cgpaSemesters
        }
      };
      downloadBlob(JSON.stringify(payload, null, 2), 'campus360_backup.json', 'application/json');
      showToast('Complete JSON backup downloaded!', 'success');
    }
  }
}

function downloadBlob(content, filename, contentType) {
  const blob = new Blob([content], { type: contentType });
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = filename;
  document.body.appendChild(a);
  a.click();
  document.body.removeChild(a);
  URL.revokeObjectURL(url);
}
```

6. public/js/attendance.js

Attendance Tracker, Safe Bunk Calculator & CSV Importer (626 lines)

```
/**
 * Campus360 - Attendance Tracker Module
 * Handles subject-wise attendance calculation, quick logs, validation, shortage estimation, and CSV bulk importer.
 */

let attendanceFilter = 'all';
let parsedCsvData = [];
let detectedCsvHeaders = [];

window.initAttendanceModule = function() {
  renderAttendanceTable();
};

// ==================== TABLE RENDERING & FILTERING ====================

function renderAttendanceTable() {
  const tbody = document.getElementById('attendanceTableBody');
  if (!tbody) return;

  const threshold = parseFloat(window.App.settings.minAttendanceThreshold) || 75;
  const margin = parseFloat(window.App.settings.warningMargin) || 5;
  const search = (document.getElementById('attendanceSearchInput')?.value || '').toLowerCase().trim();

  let list = window.App.attendance || [];

  // Filter by search query
  if (search) {
    list = list.filter(item =>
      (item.name || '').toLowerCase().includes(search) ||
      (item.code || '').toLowerCase().includes(search) ||
      (item.faculty || '').toLowerCase().includes(search)
    );
  }

  // Filter by status pill
  if (attendanceFilter !== 'all') {
    list = list.filter(item => {
      const cond = parseInt(item.conducted, 10) || 0;
      const att = parseInt(item.attended, 10) || 0;
      if (cond === 0) return attendanceFilter === 'all';
      const pct = (att / cond) * 100;
      if (attendanceFilter === 'safe') return pct >= threshold;
      if (attendanceFilter === 'warning') return pct >= (threshold - margin) && pct < threshold;
      if (attendanceFilter === 'danger') return pct < (threshold - margin);
      return true;
    });
  }

  if (list.length === 0) {
    tbody.innerHTML = `
      <tr>
        <td colspan="8" class="text-center py-6 text-muted">
          No courses matching your criteria. Click "Add Subject" or adjust search filters.
        </td>
      </tr>
    `;
    return;
  }

  tbody.innerHTML = list.map(item => {
    const cond = parseInt(item.conducted, 10) || 0;
    const att = parseInt(item.attended, 10) || 0;

    let pctText = 'Not available yet';
    let pctNum = null;
    let statusPill = `<span class="status-pill pill-neutral">Pending</span>`;
    let adviceHtml = `<span class="text-muted text-xs">No classes conducted yet</span>`;

    if (cond > 0) {
      pctNum = Math.round((att / cond) * 1000) / 10;
      pctText = `${pctNum}%`;

      if (pctNum >= threshold) {
```

```
        statusPill = `<span class="status-pill pill-safe">Safe (&ge;${threshold}%)</span>`;
        const bunks = Math.floor((100 * att - threshold * cond) / threshold);
        if (bunks > 0) {
          adviceHtml = `<span class="text-emerald-700 text-xs"><strong>${bunks} class${bunks > 1 ? 'es' : ''}</strong> can be safely missed</span>`;
        } else {
          adviceHtml = `<span class="text-xs text-muted">Borderline safe. Do not miss upcoming classes.</span>`;
        }
      } else if (pctNum >= threshold - margin) {
        statusPill = `<span class="status-pill pill-warning">Near Margin</span>`;
        const needed = Math.ceil((threshold * cond - 100 * att) / (100 - threshold));
        adviceHtml = `<span class="text-amber-800 text-xs font-semibold">Attend next <strong>${needed}</strong> class${needed > 1 ? 'es' : ''} to reach ${threshold}%</span>`;
      } else {
        statusPill = `<span class="status-pill pill-danger">Shortage (&lt;${threshold}%)</span>`;
        const needed = Math.ceil((threshold * cond - 100 * att) / (100 - threshold));
        adviceHtml = `<span class="text-rose-700 text-xs font-bold">Must attend next <strong>${needed}</strong> consecutive classes!</span>`;
      }
    }

    return `
      <tr id="row_${item.id}">
        <td>
          <div class="font-bold text-primary">${escapeHtml(item.name)}</div>
          <div class="text-xs text-muted">
            <span class="font-mono">${escapeHtml(item.code || 'N/A')}</span>
            ${item.faculty ? ` • <span>${escapeHtml(item.faculty)}</span>` : ''}
          </div>
        </td>
        <td><strong>${cond}</strong></td>
        <td><strong>${att}</strong></td>
        <td>
          <span class="font-mono font-bold text-base ${pctNum !== null && pctNum < threshold ? 'text-rose-600' : ''}">${pctText}</span>
        </td>
        <td>${statusPill}</td>
        <td>${adviceHtml}</td>
        <td>
          <div class="quick-log-group">
            <button class="quick-btn present" title="Mark +1 Present" onclick="quickLogAttendance('${item.id}', 'present')">+1 Present</button>
            <button class="quick-btn absent" title="Mark +1 Absent (Conducted increases)" onclick="quickLogAttendance('${item.id}', 'absent')">+1 Missed</button>
          </div>
        </td>
        <td>
          <div class="flex gap-1">
            <button class="btn-icon" title="Edit Subject" onclick="openEditSubjectModal('${item.id}')">
              <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M12 20h9"/><path d="M16.5 3.5a2.121 2.121 0 0 1 3 3L7 19l-4 1 1-4L16.5 3.5z"/></svg>
            </button>
            <button class="btn-icon text-danger" title="Delete Subject" onclick="confirmDeleteSubject('${item.id}', '${escapeHtml(item.name)}')">
              <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><polyline points="3 6 5 6 21 6"/><path d="M19 6v14a2 2 0 0 1-2 2H7a2 2 0 0 1-2-2V6m3 0V4a2 2 0 0 1 2-2h4a2 2 0 0 1 2 2v2"/></svg>
            </button>
          </div>
        </td>
      </tr>
    `;
  }).join('');
}

function setAttendanceFilter(filter, btn) {
  attendanceFilter = filter;
  document.querySelectorAll('.filter-pill').forEach(p => p.classList.remove('active'));
  btn.classList.add('active');
  renderAttendanceTable();
}

function filterAttendanceTable() {
  renderAttendanceTable();
}

// ==================== QUICK LOG (+1 PRESENT / +1 MISSED) ====================

async function quickLogAttendance(id, action) {
  const item = window.App.attendance.find(a => a.id === id);
  if (!item) return;

  if (window.App.isBackendConnected) {
    try {
      const res = await fetch(`${API_BASE}/attendance/${id}/increment`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ action })
```

```
      });
      if (res.ok) {
        const updated = await res.json();
        const idx = window.App.attendance.findIndex(a => a.id === id);
        if (idx !== -1) window.App.attendance[idx] = updated;
        saveToLocalStorage();
        renderAttendanceTable();
        renderDashboard();
        showToast(`Logged ${action === 'present' ? 'attendance' : 'missed class'} for ${item.name}`,
'success');
        return;
      }
    } catch (err) {
      console.warn('Backend quick log failed, falling back to local update:', err);
    }
  }

  // Local fallback
  item.conducted = (parseInt(item.conducted, 10) || 0) + 1;
  if (action === 'present') {
    item.attended = (parseInt(item.attended, 10) || 0) + 1;
  }
  saveToLocalStorage();
  renderAttendanceTable();
  renderDashboard();
  showToast(`Updated ${item.name} (${action})`, 'success');
}

// ==================== ADD / EDIT SUBJECT MODAL ====================

function openAddSubjectModal() {
  document.getElementById('subjectModalTitle').textContent = 'Add Course Attendance';
  document.getElementById('modalSubjectId').value = '';
  document.getElementById('modalSubjectCode').value = '';
  document.getElementById('modalSubjectName').value = '';
  document.getElementById('modalClassesConducted').value = '';
  document.getElementById('modalClassesAttended').value = '';
  document.getElementById('modalFaculty').value = '';
  document.getElementById('modalAttendanceError').classList.add('hidden');

  validateModalAttendance();
  document.getElementById('subjectModal').classList.remove('hidden');
}

function openEditSubjectModal(id) {
  const item = window.App.attendance.find(a => a.id === id);
  if (!item) return;

  document.getElementById('subjectModalTitle').textContent = 'Edit Course Attendance';
  document.getElementById('modalSubjectId').value = item.id;
  document.getElementById('modalSubjectCode').value = item.code || '';
  document.getElementById('modalSubjectName').value = item.name || '';
  document.getElementById('modalClassesConducted').value = item.conducted;
  document.getElementById('modalClassesAttended').value = item.attended;
  document.getElementById('modalFaculty').value = item.faculty || '';
  document.getElementById('modalAttendanceError').classList.add('hidden');

  validateModalAttendance();
  document.getElementById('subjectModal').classList.remove('hidden');
}

function closeSubjectModal() {
  document.getElementById('subjectModal').classList.add('hidden');
}

function validateModalAttendance() {
  const condInput = document.getElementById('modalClassesConducted');
  const attInput = document.getElementById('modalClassesAttended');
  const previewPct = document.getElementById('modalPreviewPct');
  const previewStatus = document.getElementById('modalPreviewStatus');
  const errBox = document.getElementById('modalAttendanceError');
  const saveBtn = document.getElementById('saveSubjectBtn');

  const cond = parseInt(condInput.value, 10);
  const att = parseInt(attInput.value, 10);
  const threshold = parseFloat(window.App.settings.minAttendanceThreshold) || 75;

  let isValid = true;
  let errorMsg = '';
```

```
  if (isNaN(cond) || cond < 0) {
    isValid = false;
  }
  if (isNaN(att) || att < 0) {
    isValid = false;
  }
  if (!isNaN(cond) && !isNaN(att) && att > cond) {
    isValid = false;
    errorMsg = 'Classes attended cannot be greater than classes conducted!';
  }

  if (errorMsg) {
    errBox.textContent = errorMsg;
    errBox.classList.remove('hidden');
    saveBtn.disabled = true;
  } else {
    errBox.classList.add('hidden');
    saveBtn.disabled = !isValid;
  }

  if (!isNaN(cond) && !isNaN(att) && cond > 0 && att <= cond) {
    const pct = Math.round((att / cond) * 1000) / 10;
    previewPct.textContent = `${pct}%`;
    if (pct >= threshold) {
      previewStatus.className = 'status-pill pill-safe';
      previewStatus.textContent = `Safe ("e ${threshold}%)`;
    } else {
      previewStatus.className = 'status-pill pill-danger';
      previewStatus.textContent = `Shortage (< ${threshold}%)`;
    }
  } else if (cond === 0 && att === 0) {
    previewPct.textContent = 'Not available yet';
    previewStatus.className = 'status-pill pill-neutral';
    previewStatus.textContent = '0 Classes Held';
  } else {
    previewPct.textContent = '--%';
    previewStatus.className = 'status-pill pill-neutral';
    previewStatus.textContent = 'Pending Input';
  }
}

async function handleSubjectFormSubmit(event) {
  event.preventDefault();

  const id = document.getElementById('modalSubjectId').value;
  const code = document.getElementById('modalSubjectCode').value.trim();
  const name = document.getElementById('modalSubjectName').value.trim();
  const conducted = parseInt(document.getElementById('modalClassesConducted').value, 10);
  const attended = parseInt(document.getElementById('modalClassesAttended').value, 10);
  const faculty = document.getElementById('modalFaculty').value.trim();

  if (!name) {
    showToast('Course name is mandatory', 'error');
    return;
  }
  if (isNaN(conducted) || conducted < 0 || isNaN(attended) || attended < 0) {
    showToast('Classes conducted and attended must be non-negative integers', 'error');
    return;
  }
  if (attended > conducted) {
    showToast('Classes attended cannot exceed classes conducted', 'error');
    return;
  }

  const payload = { code, name, conducted, attended, faculty };

  if (window.App.isBackendConnected) {
    try {
      const url = id ? `${API_BASE}/attendance/${id}` : `${API_BASE}/attendance`;
      const method = id ? 'PUT' : 'POST';

      const res = await fetch(url, {
        method,
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(payload)
      });

```

```
      if (res.ok) {
        await fetchAllFromBackend();
        closeSubjectModal();
        renderAttendanceTable();
        renderDashboard();
        showToast(id ? 'Course updated successfully' : 'Course added successfully', 'success');
        return;
      } else {
        const data = await res.json();
        showToast(data.error || 'Server rejected submission', 'error');
        return;
      }
    } catch (err) {
      console.warn('Backend submission failed, falling back to local saving:', err);
    }
  }

  // Local fallback
  if (id) {
    const idx = window.App.attendance.findIndex(a => a.id === id);
    if (idx !== -1) {
      window.App.attendance[idx] = { ...window.App.attendance[idx], ...payload };
    }
  } else {
    const newId = 'att_' + Date.now();
    window.App.attendance.push({ id: newId, ...payload });

    // Also create matching syllabus entry
    window.App.syllabus.push({
      id: 'syl_' + Date.now(),
      subjectCode: code || 'SUB-' + window.App.attendance.length,
      subjectName: name,
      totalUnits: 5,
      units: [
        { unitNo: 1, title: 'Unit 1: Fundamentals', topicsTotal: 5, topicsCompleted: 0, status: 'Not
Started'},
        { unitNo: 2, title: 'Unit 2: Core Concepts', topicsTotal: 5, topicsCompleted: 0, status: 'Not
Started'},
        { unitNo: 3, title: 'Unit 3: Intermediate Topics', topicsTotal: 5, topicsCompleted: 0, status: 'Not
Started'},
        { unitNo: 4, title: 'Unit 4: Advanced Principles', topicsTotal: 5, topicsCompleted: 0, status: 'Not
Started'},
        { unitNo: 5, title: 'Unit 5: Applications & Case Studies', topicsTotal: 5, topicsCompleted: 0,
status: 'Not Started' }
      ]
    });
  }

  saveToLocalStorage();
  closeSubjectModal();
  renderAttendanceTable();
  renderDashboard();
  showToast(id ? 'Course updated locally' : 'Course added locally', 'success');
}

function confirmDeleteSubject(id, name) {
  openConfirmModal(
    'Delete Course',
    `Are you sure you want to remove "${name}" from your attendance tracker? This action cannot be undone.`,
    async () => {
      if (window.App.isBackendConnected) {
        try {
          const res = await fetch(`${API_BASE}/attendance/${id}`, { method: 'DELETE' });
          if (res.ok) {
            await fetchAllFromBackend();
            renderAttendanceTable();
            renderDashboard();
            showToast(`Deleted ${name}`, 'success');
            return;
          }
        } catch (e) {
          console.warn('Backend delete failed:', e);
        }
      }

      window.App.attendance = window.App.attendance.filter(a => a.id !== id);
      saveToLocalStorage();
      renderAttendanceTable();
      renderDashboard();
      showToast(`Removed ${name}`, 'success');
    }
  );
}
```

```
// ==================== BULK CSV IMPORT ====================

function openBulkImportModal() {
  parsedCsvData = [];
  detectedCsvHeaders = [];
  document.getElementById('csvFileInput').value = '';
  document.getElementById('columnMappingSection').classList.add('hidden');
  document.getElementById('csvPreviewContainer').classList.add('hidden');
  document.getElementById('bulkImportError').classList.add('hidden');
  document.getElementById('confirmImportBtn').disabled = true;
  document.getElementById('bulkImportModal').classList.remove('hidden');
}

function closeBulkImportModal() {
  document.getElementById('bulkImportModal').classList.add('hidden');
}

function handleCsvFileSelected(event) {
  const file = event.target.files[0];
  if (!file) return;

  const reader = new FileReader();
  reader.onload = function(e) {
    const text = e.target.result;
    parseAndPrepareCsv(text);
  };
  reader.readAsText(file);
}

// Parse CSV text supporting comma, semicolon, tab, and quotes
function parseAndPrepareCsv(csvText) {
  const lines = csvText.split(/\r?\n/).filter(line => line.trim().length > 0);
  if (lines.length < 2) {
    showBulkImportError('CSV file must have a header row and at least one data row.');
    return;
  }

  // Detect delimiter
  const firstLine = lines[0];
  let delimiter = ',';
  if (firstLine.includes('\t')) delimiter = '\t';
  else if (firstLine.includes(';') && !firstLine.includes(',')) delimiter = ';';

  // Tokenize rows
  const parseRow = row => {
    const regex = new RegExp(`(?:^|${delimiter})(?:"([^"]*(?:""[^"]*)*)"|([^"${delimiter}]*))`, 'g');
    const result = [];
    let match;
    while ((match = regex.exec(row)) !== null) {
      let val = match[1] !== undefined ? match[1].replace(/""/g, '"') : match[2];
      result.push((val || '').trim());
      if (regex.lastIndex === match.index) regex.lastIndex++;
    }
    return result;
  };

  detectedCsvHeaders = parseRow(lines[0]);
  parsedCsvData = lines.slice(1).map(parseRow);

  populateColumnMappingDropdowns();
  document.getElementById('columnMappingSection').classList.remove('hidden');
  document.getElementById('bulkImportError').classList.add('hidden');

  previewMappedCsv();
}

function populateColumnMappingDropdowns() {
  const selects = [
    { el: document.getElementById('mapSubjectName'), hint: ['subject', 'course', 'name', 'title'] },
    { el: document.getElementById('mapSubjectCode'), hint: ['code', 'sub code', 'course code', 'id'] },
    { el: document.getElementById('mapConducted'), hint: ['conducted', 'held', 'total', 'classes conducted'] },
    { el: document.getElementById('mapAttended'), hint: ['attended', 'present', 'classes attended'] }
  ];

  selects.forEach(({ el, hint }) => {
    el.innerHTML = detectedCsvHeaders.map((h, i) => `<option value="${i}">${h} (Col ${i + 1})</option>`).join('');
    // Auto-select best match based on hint
```

```
    const matchIdx = detectedCsvHeaders.findIndex(h => hint.some(k => h.toLowerCase().includes(k)));
    if (matchIdx !== -1) el.value = matchIdx;
  });
}

function previewMappedCsv() {
  const nameIdx = parseInt(document.getElementById('mapSubjectName').value, 10);
  const codeIdx = parseInt(document.getElementById('mapSubjectCode').value, 10);
  const condIdx = parseInt(document.getElementById('mapConducted').value, 10);
  const attIdx = parseInt(document.getElementById('mapAttended').value, 10);

  const tbody = document.getElementById('csvPreviewTableBody');
  const threshold = parseFloat(window.App.settings.minAttendanceThreshold) || 75;
  const errors = [];

  let validRowsCount = 0;
  tbody.innerHTML = parsedCsvData.slice(0, 15).map((row, idx) => {
    const rowNum = idx + 1;
    const name = row[nameIdx] || '';
    const code = row[codeIdx] || `SUB-${rowNum}`;
    const conducted = parseInt(row[condIdx], 10);
    const attended = parseInt(row[attIdx], 10);

    let rowError = '';
    if (!name) rowError = 'Missing name';
    else if (isNaN(conducted) || conducted < 0) rowError = 'Invalid conducted count';
    else if (isNaN(attended) || attended < 0) rowError = 'Invalid attended count';
    else if (attended > conducted) rowError = 'Attended > Conducted';

    if (rowError) {
      errors.push(`Row ${rowNum}: ${rowError}`);
      return `
        <tr class="bg-red-50 text-danger">
          <td>${rowNum}</td>
          <td>${escapeHtml(code)}</td>
          <td>${escapeHtml(name)}</td>
          <td>${isNaN(conducted) ? '-' : conducted}</td>
          <td>${isNaN(attended) ? '-' : attended}</td>
          <td colspan="2"><span class="status-pill pill-danger">Error: ${rowError}</span></td>
        </tr>
      `;
    }

    validRowsCount++;
    const pct = conducted > 0 ? `${Math.round((attended / conducted) * 1000) / 10}%` : 'N/A';
    const statusClass = (conducted > 0 && (attended / conducted) * 100 >= threshold) ? 'pill-safe' : 'pill-
danger';
    return `
      <tr>
        <td>${rowNum}</td>
        <td><code>${escapeHtml(code)}</code></td>
        <td><strong>${escapeHtml(name)}</strong></td>
        <td>${conducted}</td>
        <td>${attended}</td>
        <td><strong>${pct}</strong></td>
        <td><span class="status-pill ${statusClass}">${(attended / conducted) * 100 >= threshold ? 'Safe' :
'Shortage'}</span></td>
      </tr>
    `;
  }).join('');

  document.getElementById('csvPreviewContainer').classList.remove('hidden');

  const confirmBtn = document.getElementById('confirmImportBtn');
  if (validRowsCount > 0) {
    confirmBtn.disabled = false;
    document.getElementById('bulkImportError').classList.add('hidden');
  } else {
    confirmBtn.disabled = true;
    showBulkImportError('No valid rows could be mapped with the selected columns.');
  }
}

async function executeBulkImport() {
  const nameIdx = parseInt(document.getElementById('mapSubjectName').value, 10);
  const codeIdx = parseInt(document.getElementById('mapSubjectCode').value, 10);
  const condIdx = parseInt(document.getElementById('mapConducted').value, 10);
  const attIdx = parseInt(document.getElementById('mapAttended').value, 10);
  const replaceExisting = document.getElementById('replaceExistingCheckbox').checked;
```

```
  const validRecords = [];
  parsedCsvData.forEach((row, idx) => {
    const name = (row[nameIdx] || '').trim();
    const code = (row[codeIdx] || `SUB-${idx + 1}`).trim();
    const conducted = parseInt(row[condIdx], 10);
    const attended = parseInt(row[attIdx], 10);

    if (name && !isNaN(conducted) && conducted >= 0 && !isNaN(attended) && attended >= 0 && attended <= conducted) {
      validRecords.push({
        id: 'att_' + Date.now() + '_' + idx,
        code,
        name,
        conducted,
        attended,
        faculty: '',
        lastUpdated: new Date().toISOString().split('T')[0]
      });
    }
  });

  if (validRecords.length === 0) {
    showBulkImportError('Could not find any valid records to import.');
    return;
  }

  if (window.App.isBackendConnected) {
    try {
      const res = await fetch(`${API_BASE}/attendance/bulk`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ items: validRecords, replaceExisting })
      });
      if (res.ok) {
        await fetchAllFromBackend();
        closeBulkImportModal();
        renderAttendanceTable();
        renderDashboard();
        showToast(`Successfully imported ${validRecords.length} courses!`, 'success');
        return;
      }
    } catch (err) {
      console.warn('Backend bulk import failed, falling back to local merge:', err);
    }
  }

  // Local fallback
  if (replaceExisting) {
    window.App.attendance = validRecords;
  } else {
    validRecords.forEach(newItem => {
      const existing = window.App.attendance.find(a => a.name.toLowerCase() === newItem.name.toLowerCase() || a.code.toLowerCase() === newItem.code.toLowerCase());
      if (existing) {
        existing.conducted = newItem.conducted;
        existing.attended = newItem.attended;
      } else {
        window.App.attendance.push(newItem);
      }
    });
  }

  saveToLocalStorage();
  closeBulkImportModal();
  renderAttendanceTable();
  renderDashboard();
  showToast(`Imported ${validRecords.length} courses locally!`, 'success');
}

function showBulkImportError(msg) {
  const errBox = document.getElementById('bulkImportError');
  errBox.textContent = msg;
  errBox.classList.remove('hidden');
}

function escapeHtml(str) {
  if (!str) return '';
  return str.toString()
    .replace(/&/g, '&amp;')
```

```
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;')
    .replace(/'/g, '&#039;');
}
```

7. public/js/syllabus.js

Syllabus Completion & Topic Tracker (JavaScript) (266 lines)

```
/**
 * Campus360 - Syllabus Completion Tracker Module
 * Calculates subject-wise unit & topic coverage, completion percentages, and milestone progress.
 */

window.initSyllabusModule = function() {
  renderSyllabusCards();
};

function renderSyllabusCards() {
  const container = document.getElementById('syllabusCardsContainer');
  const heroPct = document.getElementById('syllabusHeroPct');
  const heroFill = document.getElementById('syllabusHeroFill');
  const heroTopics = document.getElementById('syllabusHeroTopics');
  const heroStatus = document.getElementById('syllabusHeroStatus');

  if (!container) return;

  const syllabusList = window.App.syllabus || [];

  // Calculate overall metrics
  let grandTotalTopics = 0;
  let grandCompletedTopics = 0;

  syllabusList.forEach(item => {
    (item.units || []).forEach(u => {
      grandTotalTopics += (parseInt(u.topicsTotal, 10) || 0);
      grandCompletedTopics += (parseInt(u.topicsCompleted, 10) || 0);
    });
  });

  const overallPct = grandTotalTopics > 0
    ? Math.round((grandCompletedTopics / grandTotalTopics) * 100)
    : 0;

  if (heroPct) heroPct.textContent = `${overallPct}%`;
  if (heroFill) heroFill.style.width = `${overallPct}%`;
  if (heroTopics) heroTopics.textContent = `${grandCompletedTopics} / ${grandTotalTopics} Total Topics
Covered`;
  if (heroStatus) {
    if (overallPct >= 80) heroStatus.textContent = 'Overall Status: Exam Ready / Ahead';
    else if (overallPct >= 50) heroStatus.textContent = 'Overall Status: On Track';
    else heroStatus.textContent = 'Overall Status: Needs Urgent Review';
  }

  if (syllabusList.length === 0) {
    container.innerHTML = `
      <div class="card full-width text-center py-8 text-muted">
        <h3>No course syllabus records tracked yet</h3>
        <p class="mt-2 text-sm">Add a subject from the Attendance tab or click "Add Subject Syllabus"
above.</p>
      </div>
    `;
    return;
  }

  container.innerHTML = syllabusList.map(item => {
    let subTotal = 0;
    let subComp = 0;
    const units = item.units || [];

    units.forEach(u => {
      subTotal += (parseInt(u.topicsTotal, 10) || 0);
      subComp += (parseInt(u.topicsCompleted, 10) || 0);
    });

    const subPct = subTotal > 0 ? Math.round((subComp / subTotal) * 100) : 0;
    let statusClass = 'pill-neutral';
    let statusLabel = 'Not Started';

    if (subPct === 100) {
      statusClass = 'pill-safe';
      statusLabel = 'Completed';
    } else if (subPct >= 60) {
      statusClass = 'pill-safe';
```

```
      statusLabel = 'On Track';
    } else if (subPct > 0) {
      statusClass = 'pill-warning';
      statusLabel = 'In Progress';
    }

    const unitsHtml = units.map(u => {
      const tot = parseInt(u.topicsTotal, 10) || 0;
      const comp = parseInt(u.topicsCompleted, 10) || 0;
      const uPct = tot > 0 ? Math.round((comp / tot) * 100) : 0;
      const isDone = uPct === 100;

      return `
        <div class="unit-item">
          <div class="unit-item-top">
            <span class="unit-title-text">${escapeHtml(u.title || `Unit ${u.unitNo}`)}</span>
            <div class="unit-topic-controls">
              <button class="unit-btn" title="Decrease completed topic"
onclick="adjustUnitProgress('${item.id}', ${u.unitNo}, -1)">&minus;</button>
              <span class="font-mono text-xs" title="Unit Hold px-1">${comp}/${tot}</span>
              <button class="unit-btn" title="Increase completed topic"
onclick="adjustUnitProgress('${item.id}', ${u.unitNo}, 1)">&#43;</button>
              <button class="btn-icon" title="Edit Unit Title" onclick="openEditUnitModal('${item.id}',
${u.unitNo}, '${escapeHtml(u.title || '')}', ${tot}, ${comp})">
<svg width="12" height="12" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2"><path d="M12 20h9"/><path d="M16.5 3.5a2.121 2.121 0 0 1 3 3L7 19l-4 1 1-4L16.5 3.5z"/></svg>
              </button>
            </div>
          </div>
          <div class="unit-progress-track">
            <div class="unit-progress-fill ${isDone ? 'done' : ''}" style="width: ${uPct}%"></div>
          </div>
          <div class="flex justify-between text-xs text-muted">
            <span>${uPct}% Finished</span>
            <span>${u.status || (isDone ? 'Completed' : (comp > 0 ? 'In Progress' : 'Not Started'))}</span>
          </div>
        </div>
      `;
    }).join('');

    return `
      <div class="syllabus-card" id="syl_card_${item.id}">
        <div>
          <div class="syllabus-card-header">
            <div>
              <span class="syllabus-subject-code">${escapeHtml(item.subjectCode || 'SUB')}</span>
              <h3 class="syllabus-subject-title">${escapeHtml(item.subjectName)}</h3>
            </div>
            <span class="status-pill ${statusClass}">${statusLabel}</span>
          </div>

          <div class="flex justify-between items-baseline mb-1">
            <span class="text-xs text-muted">Subject Coverage</span>
            <span class="font-bold text-sm">${subPct}% (${subComp}/${subTotal} Topics)</span>
          </div>

          <div class="progress-bar-track mb-3">
            <div class="progress-bar-fill progress-blue" style="width: ${subPct}%"></div>
          </div>

          <div class="unit-list">
            ${unitsHtml}
          </div>
        </div>

        <div class="pt-2 border-t border-slate-100 flex justify-between items-center">
          <button class="btn btn-sm btn-outline" onclick="openAddUnitModal('${item.id}')">+ Add Unit</
button>
          <span class="text-xs text-muted">${units.length} Modules Total</span>
        </div>
      </div>
    `;
  }).join('');
}

// Quick Topic +1 / -1
async function adjustUnitProgress(subjectId, unitNo, delta) {
  const syl = window.App.syllabus.find(s => s.id === subjectId);
  if (!syl) return;

  const unit = syl.units.find(u => u.unitNo === unitNo);
  if (!unit) return;

```

```
  const tot = parseInt(unit.topicsTotal, 10) || 0;
  let comp = parseInt(unit.topicsCompleted, 10) || 0;
  comp = Math.max(0, Math.min(tot, comp + delta));
  unit.topicsCompleted = comp;

  if (comp === tot) unit.status = 'Completed';
  else if (comp > 0) unit.status = 'In Progress';
  else unit.status = 'Not Started';

  if (window.App.isBackendConnected) {
    try {
      await fetch(`${API_BASE}/syllabus/${subjectId}/unit/${unitNo}`, {
        method: 'PUT',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ topicsCompleted: comp, status: unit.status })
      });
    } catch (e) {
      console.warn('Backend sync failed for syllabus unit:', e);
    }
  }

  saveToLocalStorage();
  renderSyllabusCards();
  renderDashboard();
}

// ==================== UNIT MODAL ====================

function openAddUnitModal(subjectId) {
  const syl = window.App.syllabus.find(s => s.id === subjectId);
  if (!syl) return;

  const nextUnitNo = (syl.units?.length || 0) + 1;
  document.getElementById('syllabusModalTitle').textContent = `Add Unit to ${syl.subjectName}`;
  document.getElementById('sylSubjectId').value = subjectId;
  document.getElementById('sylUnitNo').value = nextUnitNo;
  document.getElementById('sylUnitTitle').value = `Unit ${nextUnitNo}: New Module`;
  document.getElementById('sylTopicsTotal').value = 5;
  document.getElementById('sylTopicsCompleted').value = 0;
  document.getElementById('sylUnitStatus').value = 'Not Started';

  document.getElementById('syllabusUnitModal').classList.remove('hidden');
}

function openEditUnitModal(subjectId, unitNo, title, total, comp) {
  document.getElementById('syllabusModalTitle').textContent = `Edit Unit ${unitNo}`;
  document.getElementById('sylSubjectId').value = subjectId;
  document.getElementById('sylUnitNo').value = unitNo;
  document.getElementById('sylUnitTitle').value = title;
  document.getElementById('sylTopicsTotal').value = total;
  document.getElementById('sylTopicsCompleted').value = comp;
  document.getElementById('sylUnitStatus').value = comp >= total ? 'Completed' : (comp > 0 ? 'In
Progress' : 'Not Started');
  document.getElementById('syllabusUnitModal').classList.remove('hidden');
}

function closeSyllabusModal() {
  document.getElementById('syllabusUnitModal').classList.add('hidden');
}

async function handleSyllabusUnitSubmit(event) {
  event.preventDefault();

  const subjectId = document.getElementById('sylSubjectId').value;
  const unitNo = parseInt(document.getElementById('sylUnitNo').value, 10);
  const title = document.getElementById('sylUnitTitle').value.trim();
  const topicsTotal = parseInt(document.getElementById('sylTopicsTotal').value, 10);
  const topicsCompleted = parseInt(document.getElementById('sylTopicsCompleted').value, 10);
  const status = document.getElementById('sylUnitStatus').value;

  const syl = window.App.syllabus.find(s => s.id === subjectId);
  if (!syl) return;

  let unit = (syl.units || []).find(u => u.unitNo === unitNo);
  if (unit) {
    unit.title = title;
    unit.topicsTotal = topicsTotal;
    unit.topicsCompleted = Math.min(topicsTotal, topicsCompleted);
```

```
    unit.status = status;
  } else {
    if (!syl.units) syl.units = [];
    syl.units.push({
      unitNo,
      title,
      topicsTotal,
      topicsCompleted: Math.min(topicsTotal, topicsCompleted),
      status
    });
  }

  if (window.App.isBackendConnected) {
    try {
      await fetch(`${API_BASE}/syllabus/${subjectId}`, {
        method: 'PUT',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(syl)
      });
    } catch (e) {
      console.warn('Backend update failed for syllabus:', e);
    }
  }

  saveToLocalStorage();
  closeSyllabusModal();
  renderSyllabusCards();
  renderDashboard();
  showToast('Unit updated successfully!', 'success');
}

function openAddSyllabusSubjectModal() {
  // Shortcut to add subject in attendance which automatically initializes syllabus
  openAddSubjectModal();
}
```

8. public/js/gpa.js

Semester GPA (SGPA) Dynamic Calculator (JavaScript) (254 lines)

```
/**
 * Campus360 - Semester GPA Calculator Module
 * Calculates weighted SGPA based on subject credits and grade points using configurable grading scales.
 */

window.initGpaModule = function() {
  if (!window.App.gpaCourses || window.App.gpaCourses.length === 0) {
    // Populate from attendance subjects or defaults
    populateDefaultGpaCourses();
  }
  renderGpaTable();
};

function populateDefaultGpaCourses() {
  if (window.App.attendance && window.App.attendance.length > 0) {
    window.App.gpaCourses = window.App.attendance.map((att, i) => ({
      id: 'gpa_' + Date.now() + '_' + i,
      code: att.code || `SUB-${i + 1}`,
      name: att.name,
      credits: 4,
      grade: 'A',
      points: 8
    }));
  } else {
    window.App.gpaCourses = [
      { id: 'gpa_1', code: 'CS301', name: 'Mathematics III', credits: 4, grade: 'A', points: 8 },
      { id: 'gpa_2', code: 'CS302', name: 'Physics & Engineering Electromagnetics', credits: 4, grade:
'B+', points: 7 },
      { id: 'gpa_3', code: 'CS303', name: 'Programming in C/C++', credits: 3, grade: 'O', points: 10 },
      { id: 'gpa_4', code: 'CS304', name: 'Data Structures & Algorithms', credits: 4, grade: 'A+', points:
9 } ];
  }
}

function renderGpaTable() {
  const tbody = document.getElementById('gpaTableBody');
  if (!tbody) return;

  const gradeScale = window.App.settings.gradeScale || [];

  if (window.App.gpaCourses.length === 0) {
    tbody.innerHTML = `
      <tr>
        <td colspan="7" class="text-center py-6 text-muted">
          No courses entered. Click "Add Course Row" to start calculating your Semester GPA.
        </td>
      </tr>
    `;
    calculateGpaMetrics();
    return;
  }

  tbody.innerHTML = window.App.gpaCourses.map((c, idx) => {
    // Generate dropdown options from settings gradeScale
    let gradeOptions = `<option value="">Select Grade</option>`;
    gradeScale.forEach(gs => {
      const isSelected = (c.grade || '').toUpperCase() === gs.grade.toUpperCase();
      gradeOptions += `<option value="${gs.grade}" data-points="${gs.points}" ${isSelected ? 'selected' :
''}>${gs.grade} (${gs.points} pts)</option>`;
    });

    const creditVal = parseFloat(c.credits) || 0;
    const pointVal = parseFloat(c.points) || 0;
    const creditPoints = (creditVal * pointVal).toFixed(1);

    return `
      <tr id="gpa_row_${c.id}">
        <td>
          <input type="text" class="form-input font-mono text-xs" style="width: 100px"
value="${escapeHtml(c.code || '')}" placeholder="Code" oninput="updateGpaCourseField('${c.id}', 'code',
this.value)">
        </td>
        <td>
          <input type="text" class="form-input" style="min-width: 180px" value="${escapeHtml(c.name ||
'')}" placeholder="Course Name" oninput="updateGpaCourseField('${c.id}', 'name', this.value)">
        </td>
        <td>
          <input type="number" class="form-input font-bold" style="width: 80px" min="0" max="15" step="0.5"
```

```
value="${c.credits}" oninput="updateGpaCourseField('${c.id}', 'credits', this.value)">
```

```
        </td>
        <td>
          <select class="form-select font-semibold" style="width: 130px"
onchange="handleGpaGradeChange('${c.id}', this)">
            ${gradeOptions}
          </select>
        </td>
        <td>
          <input type="number" class="form-input font-mono" style="width: 80px" min="0" max="10" step="0.1"
value="${c.points}" oninput="updateGpaCourseField('${c.id}', 'points', this.value)">
        </td>
        <td>
          <span class="font-mono font-bold text-primary" id="cp_${c.id}">${creditPoints}</span>
        </td>
        <td>
          <button class="btn-icon text-danger" title="Remove course" onclick="removeGpaRow('${c.id}')">
            <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2"><polyline points="3 6 5 6 21 6"/><path d="M19 6v14a2 2 0 0 1-2 2H7a2 2 0 0 1-2-2V6m3 0V4a2 2 0 0
1 2-2h4a2 2 0 0 1 2 2v2"/></svg>
          </button>
        </td>
      </tr>
    `;
  }).join('');

  calculateGpaMetrics();
}

function handleGpaGradeChange(courseId, selectEl) {
  const selectedGrade = selectEl.value;
  const selectedOption = selectEl.options[selectEl.selectedIndex];
  const points = selectedOption ? parseFloat(selectedOption.getAttribute('data-points')) : 0;

  const course = window.App.gpaCourses.find(c => c.id === courseId);
  if (course) {
    course.grade = selectedGrade;
    course.points = isNaN(points) ? 0 : points;
  }

  // Update input row without full re-render to keep focus
  const row = document.getElementById(`gpa_row_${courseId}`);
  if (row) {
    const ptInput = row.querySelector('input[type="number"][min="0"][max="10"]');
    if (ptInput) ptInput.value = course.points;
    const cpSpan = document.getElementById(`cp_${courseId}`);
    if (cpSpan) cpSpan.textContent = ((parseFloat(course.credits) || 0) * course.points).toFixed(1);
  }

  calculateGpaMetrics();
}

function updateGpaCourseField(courseId, field, val) {
  const course = window.App.gpaCourses.find(c => c.id === courseId);
  if (!course) return;

  if (field === 'credits') {
    course.credits = Math.max(0, parseFloat(val) || 0);
  } else if (field === 'points') {
    course.points = Math.max(0, parseFloat(val) || 0);
  } else {
    course[field] = val;
  }

  const cpSpan = document.getElementById(`cp_${courseId}`);
  if (cpSpan) {
    const cred = parseFloat(course.credits) || 0;
    const pt = parseFloat(course.points) || 0;
    cpSpan.textContent = (cred * pt).toFixed(1);
  }

  calculateGpaMetrics();
}

function calculateGpaMetrics() {
  let totalCredits = 0;
  let totalPoints = 0;

  window.App.gpaCourses.forEach(c => {
    // Only include courses that have a defined grade or points
    const cred = parseFloat(c.credits) || 0;
    const pt = parseFloat(c.points) || 0;
    if (c.grade || pt > 0) {
```

```
      totalCredits += cred;
      totalPoints += (cred * pt);
    }
  });

  const sgpa = totalCredits > 0 ? (Math.round((totalPoints / totalCredits) * 100) / 100).toFixed(2) :
'0.00';
  const gpaNumberEl = document.getElementById('calculatedGpa');
  const totalCredsEl = document.getElementById('gpaTotalCredits');
  const totalPointsEl = document.getElementById('gpaTotalPoints');

  if (gpaNumberEl) gpaNumberEl.textContent = sgpa;
  if (totalCredsEl) totalCredsEl.textContent = totalCredits;
  if (totalPointsEl) totalPointsEl.textContent = totalPoints.toFixed(2);

  // Sync to dashboard
  const kpiGpa = document.getElementById('kpiSemesterGPA');
  const kpiGpaCredits = document.getElementById('kpiGpaCredits');
  if (kpiGpa) kpiGpa.textContent = sgpa;
  if (kpiGpaCredits) kpiGpaCredits.textContent = `${totalCredits} registered credits`;
}

function addGpaRow() {
  const newRow = {
    id: 'gpa_' + Date.now(),
    code: 'CS' + (300 + window.App.gpaCourses.length + 1),
    name: 'New Course ' + (window.App.gpaCourses.length + 1),
    credits: 3,
    grade: 'A',
    points: 8
  };
  window.App.gpaCourses.push(newRow);
  renderGpaTable();
  showToast('Added course row', 'info');
}

function removeGpaRow(courseId) {
  window.App.gpaCourses = window.App.gpaCourses.filter(c => c.id !== courseId);
  renderGpaTable();
  showToast('Removed course row', 'info');
}

function resetGpaForm() {
  openConfirmModal('Reset GPA Calculator', 'Clear all rows and restore course list from your attendance
timetable?', () => {
    populateDefaultGpaCourses();
    renderGpaTable();
    showToast('Reset GPA courses', 'info');
  });
}

async function saveGpaToBackend() {
  if (window.App.isBackendConnected) {
    try {
      const res = await fetch(`${API_BASE}/gpa`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ courses: window.App.gpaCourses })
      });
      if (res.ok) {
        showToast('Semester GPA saved to database!', 'success');
        saveToLocalStorage();
        renderDashboard();
        return;
      }
    } catch (e) {
      console.warn('Backend save failed:', e);
    }
  }

  saveToLocalStorage();
  renderDashboard();
  showToast('Semester GPA saved locally!', 'success');
}

function transferGpaToCgpa() {
  calculateGpaMetrics();
  const sgpa = parseFloat(document.getElementById('calculatedGpa').textContent) || 0;
  const credits = parseFloat(document.getElementById('gpaTotalCredits').textContent) || 0;
```

```
  if (credits === 0) {
    showToast('Add course credits before pushing to CGPA', 'error');
    return;
  }

  // Find or create current semester entry in CGPA
  let curSem = window.App.cgpaSemesters.find(s => s.semester.includes('Current') || s.semester === 'Semester 4');
  if (curSem) {
    curSem.credits = credits;
    curSem.sgpa = sgpa;
  } else {
    window.App.cgpaSemesters.push({
      id: 'sem_' + Date.now(),
      semester: `Semester ${window.App.cgpaSemesters.length + 1} (Current)`,
      credits,
      sgpa
    });
  }

  switchTab('cgpa');
  showToast(`SGPA (${sgpa}) transferred to CGPA Calculator!`, 'success');
}
```

9. public/js/cgpa.js

Cumulative CGPA Calculator & Target Planner (JavaScript) (241 lines)

```
/**
 * Campus360 - Cumulative CGPA Calculator & Graduation Planner Module
 * Calculates credit-weighted CGPA across multiple semesters and forecasts required SGPA for academic goals.
 */

window.initCgpaModule = function() {
  if (!window.App.cgpaSemesters || window.App.cgpaSemesters.length === 0) {
    window.App.cgpaSemesters = [
      { id: 'sem_1', semester: 'Semester 1', credits: 20, sgpa: 8.40 },
      { id: 'sem_2', semester: 'Semester 2', credits: 22, sgpa: 8.65 },
      { id: 'sem_3', semester: 'Semester 3', credits: 21, sgpa: 8.10 },
      { id: 'sem_4', semester: 'Semester 4 (Current)', credits: 20, sgpa: 8.25 }
    ];
  }
  renderCgpaTable();
};

function renderCgpaTable() {
  const tbody = document.getElementById('cgpaTableBody');
  if (!tbody) return;

  if (window.App.cgpaSemesters.length === 0) {
    tbody.innerHTML = `
      <tr>
        <td colspan="5" class="text-center py-6 text-muted">
          No semester records added yet. Click "Add Semester" to begin.
        </td>
      </tr>
    `;
    calculateCgpaMetrics();
    return;
  }

  tbody.innerHTML = window.App.cgpaSemesters.map((s, idx) => {
    const cred = parseFloat(s.credits) || 0;
    const sgpa = parseFloat(s.sgpa) || 0;
    const weightedPts = (cred * sgpa).toFixed(2);

    return `
      <tr id="sem_row_${s.id}">
        <td>
          <input type="text" class="form-input font-bold" style="min-width: 160px"
value="${escapeHtml(s.semester)}" placeholder="e.g. Semester ${idx + 1}"
oninput="updateCgpaSemesterField('${s.id}', 'semester', this.value)">
        </td>
        <td>
          <input type="number" class="form-input font-bold" style="width: 100px" min="0" max="40" step="1"
value="${s.credits}" oninput="updateCgpaSemesterField('${s.id}', 'credits', this.value)">
        </td>
        <td>
          <input type="number" class="form-input font-mono font-bold" style="width: 110px" min="0" max="10"
step="0.01" value="${s.sgpa}" oninput="updateCgpaSemesterField('${s.id}', 'sgpa', this.value)">
        </td>
        <td>
          <span class="font-mono font-bold text-teal-700" id="sem_wp_${s.id}">${weightedPts}</span>
        </td>
        <td>
          <button class="btn-icon text-danger" title="Remove Semester"
onclick="removeCgpaSemesterRow('${s.id}')">
<svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2"><polyline points="3 6 5 6 21 6"/><path d="M19 6v14a2 2 0 0 1-2 2H7a2 2 0 0 1-2-2V6m3 0V4a2 2 0 0
1 2-2h4a2 2 0 0 1 2 2v2"/></svg>
          </button>
        </td>
      </tr>
    `;
  }).join('');

  calculateCgpaMetrics();
}

function updateCgpaSemesterField(semId, field, val) {
  const sem = window.App.cgpaSemesters.find(s => s.id === semId);
  if (!sem) return;

  if (field === 'credits') {
    sem.credits = Math.max(0, parseFloat(val) || 0);
  } else if (field === 'sgpa') {
    sem.sgpa = Math.max(0, Math.min(10, parseFloat(val) || 0));
  } else {
```

```
    sem[field] = val;
  }

  const wpSpan = document.getElementById(`sem_wp_${semId}`);
  if (wpSpan) {
    const cred = parseFloat(sem.credits) || 0;
    const sgpa = parseFloat(sem.sgpa) || 0;
    wpSpan.textContent = (cred * sgpa).toFixed(2);
  }

  calculateCgpaMetrics();
}

function calculateCgpaMetrics() {
  let totalCredits = 0;
  let totalWeightedPts = 0;

  window.App.cgpaSemesters.forEach(s => {
    const cred = parseFloat(s.credits) || 0;
    const sgpa = parseFloat(s.sgpa) || 0;
    totalCredits += cred;
    totalWeightedPts += (cred * sgpa);
  });

  const cgpaVal = totalCredits > 0 ? (Math.round((totalWeightedPts / totalCredits) * 100) / 100) : 0;
  const cgpaFormatted = cgpaVal.toFixed(2);

  // Conversion rule (Standard AICTE/UGC formula: Percentage = (CGPA - 0.75) * 10 or CGPA * 9.5)
  const approxPercentage = cgpaVal > 0 ? Math.max(0, ((cgpaVal - 0.75) * 10)).toFixed(1) : '0.0';

  const cgpaNumberEl = document.getElementById('calculatedCgpa');
  const totalCredsEl = document.getElementById('cgpaTotalCredits');
  const totalPtsEl = document.getElementById('cgpaTotalPoints');
  const pctEl = document.getElementById('cgpaPercentage');
  const classEl = document.getElementById('cgpaClassification');

  if (cgpaNumberEl) cgpaNumberEl.textContent = cgpaFormatted;
  if (totalCredsEl) totalCredsEl.textContent = totalCredits;
  if (totalPtsEl) totalPtsEl.textContent = totalWeightedPts.toFixed(2);
  if (pctEl) pctEl.textContent = `${approxPercentage}%`;

  if (classEl) {
    if (cgpaVal >= 8.0) {
      classEl.textContent = 'Ø<ß  First Class with Distinction';
      classEl.className = 'cgpa-classification bg-emerald-50 text-emerald-800 border-emerald-200';
    } else if (cgpaVal >= 6.75) {
      classEl.textContent = ''  First Class';
      classEl.className = 'cgpa-classification bg-blue-50 text-blue-800 border-blue-200';
    } else if (cgpaVal >= 5.5) {
      classEl.textContent = 'Ø=Ý9 Second Class';
      classEl.className = 'cgpa-classification bg-amber-50 text-amber-800 border-amber-200';
    } else if (cgpaVal >= 4.0) {
      classEl.textContent = 'Ø=Ý8 Pass Class';
      classEl.className = 'cgpa-classification bg-slate-50 text-slate-800 border-slate-200';
    } else {
      classEl.textContent = '& þ  Remedial / Arrears';
      classEl.className = 'cgpa-classification bg-rose-50 text-rose-800 border-rose-200';
    }
  }

  // Sync to dashboard
  const kpiCgpa = document.getElementById('kpiCumulativeCGPA');
  const kpiCgpaCredits = document.getElementById('kpiCgpaCredits');
  if (kpiCgpa) kpiCgpa.textContent = cgpaFormatted;
  if (kpiCgpaCredits) kpiCgpaCredits.textContent = `${totalCredits} total earned credits`;
}

function addCgpaSemesterRow() {
  const nextNum = window.App.cgpaSemesters.length + 1;
  const newRow = {
    id: 'sem_' + Date.now(),
    semester: `Semester ${nextNum}`,
    credits: 20,
    sgpa: 8.00
  };
  window.App.cgpaSemesters.push(newRow);
  renderCgpaTable();
  showToast(`Added Semester ${nextNum}`, 'info');
```

```
}

function removeCgpaSemesterRow(semId) {
  window.App.cgpaSemesters = window.App.cgpaSemesters.filter(s => s.id !== semId);
  renderCgpaTable();
  showToast('Removed semester row', 'info');
}

async function saveCgpaToBackend() {
  if (window.App.isBackendConnected) {
    try {
      const res = await fetch(`${API_BASE}/cgpa`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ semesters: window.App.cgpaSemesters })
      });
      if (res.ok) {
        showToast('CGPA records saved to database!', 'success');
        saveToLocalStorage();
        renderDashboard();
        return;
      }
    } catch (e) {
      console.warn('Backend CGPA save failed:', e);
    }
  }

  saveToLocalStorage();
  renderDashboard();
  showToast('CGPA records saved locally!', 'success');
}

// ==================== TARGET CGPA PLANNER ====================

function openTargetCgpaModal() {
  calculateTargetPlanner();
  document.getElementById('targetCgpaModal').classList.remove('hidden');
}

function closeTargetCgpaModal() {
  document.getElementById('targetCgpaModal').classList.add('hidden');
}

function calculateTargetPlanner() {
  const targetCgpa = parseFloat(document.getElementById('plannerTargetCgpa').value) || 8.5;
  const futureSemesters = parseInt(document.getElementById('plannerFutureSemesters').value, 10) || 4;
  const avgCredits = parseFloat(document.getElementById('plannerAvgCredits').value) || 20;

  let currentEarnedCredits = 0;
  let currentWeightedPts = 0;

  window.App.cgpaSemesters.forEach(s => {
    const cred = parseFloat(s.credits) || 0;
    const sgpa = parseFloat(s.sgpa) || 0;
    currentEarnedCredits += cred;
    currentWeightedPts += (cred * sgpa);
  });

  const futureTotalCredits = futureSemesters * avgCredits;
  const grandTotalCredits = currentEarnedCredits + futureTotalCredits;

  const targetTotalPts = targetCgpa * grandTotalCredits;
  const neededFuturePts = targetTotalPts - currentWeightedPts;
  const requiredSgpa = futureTotalCredits > 0 ? (neededFuturePts / futureTotalCredits) : 0;

  const resVal = document.getElementById('plannerRequiredSgpa');
  const resNote = document.getElementById('plannerNote');

  if (requiredSgpa <= 10.0 && requiredSgpa >= 0) {
    resVal.textContent = requiredSgpa.toFixed(2);
    resVal.className = 'planner-value text-emerald-700';
    if (requiredSgpa <= 8.0) {
      resNote.textContent = 'Ø<ß‰ Very achievable! Maintain consistent preparation.';
    } else if (requiredSgpa <= 9.0) {
      resNote.textContent = 'Ø=Üª Strong target! Focus on high-credit core subjects.';
    } else {
      resNote.textContent = 'Ø=Ý% High performance needed! Aim for top grades in all subjects.';
    }
```

```
  } else if (requiredSgpa > 10.0) {
    resVal.textContent = `${requiredSgpa.toFixed(2)} (Exceeds 10.0)`;
    resVal.className = 'planner-value text-rose-600';
    resNote.textContent = '& þ  Mathematically impossible with remaining semesters alone. Adjust target or add elective credits.';
  } else {
    resVal.textContent = 'Achieved';
    resVal.className = 'planner-value text-teal-700';
    resNote.textContent = ''( You have already satisfied or exceeded this target!';
  }
}
```

10. public/js/settings.js

Institutional Rules, Grading Scales & Data Backups (227 lines)

```
/**
 * Campus360 - Settings & Institutional Configuration Module
 * Manages attendance thresholds, warning margins, grading scale maps, JSON/CSV backups, and data resets.
 */

window.initSettingsModule = function() {
  renderSettingsForm();
};

function renderSettingsForm() {
  const threshold = window.App.settings.minAttendanceThreshold || 75;
  const margin = window.App.settings.warningMargin || 5;
  const rule = window.App.settings.institutionRule || 'weighted_credits';

  const rangeEl = document.getElementById('settingThresholdRange');
  const numEl = document.getElementById('settingMinThreshold');
  const marginEl = document.getElementById('settingWarningMargin');
  const ruleEl = document.getElementById('settingInstitutionRule');

  if (rangeEl) rangeEl.value = threshold;
  if (numEl) numEl.value = threshold;
  if (marginEl) marginEl.value = margin;
  if (ruleEl) ruleEl.value = rule;

  renderGradeScaleTable();
}

function syncThresholdRange(val) {
  document.getElementById('settingMinThreshold').value = val;
}

function syncThresholdInput(val) {
  const v = Math.max(50, Math.min(100, parseFloat(val) || 75));
  document.getElementById('settingThresholdRange').value = v;
}

function renderGradeScaleTable() {
  const tbody = document.getElementById('gradeScaleTableBody');
  if (!tbody) return;

  const scale = window.App.settings.gradeScale || [];

  tbody.innerHTML = scale.map((item, idx) => `
    <tr>
      <td>
        <input type="text" class="form-input font-bold" style="width: 80px"
value="${escapeHtml(item.grade)}" oninput="updateGradeScaleItem(${idx}, 'grade', this.value)">
      </td>
      <td>
        <input type="number" class="form-input font-mono font-bold" style="width: 90px" min="0" max="10"
step="0.5" value="${item.points}" oninput="updateGradeScaleItem(${idx}, 'points', this.value)">
      </td>
      <td>
        <input type="text" class="form-input" value="${escapeHtml(item.description || '')}"
placeholder="Description" oninput="updateGradeScaleItem(${idx}, 'description', this.value)">
      </td>
      <td>
        <button class="btn-icon text-danger" title="Delete Grade" onclick="removeGradeScaleRow(${idx})">
          <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2"><polyline points="3 6 5 6 21 6"/><path d="M19 6v14a2 2 0 0 1-2 2H7a2 2 0 0 1-2-2V6m3 0V4a2 2 0 0
1 2-2h4a2 2 0 0 1 2 2v2"/></svg>
        </button>
      </td>
    </tr>
  `).join('');
}

function updateGradeScaleItem(index, field, val) {
  const scale = window.App.settings.gradeScale;
  if (!scale || !scale[index]) return;

  if (field === 'points') {
    scale[index].points = Math.max(0, parseFloat(val) || 0);
  } else {
    scale[index][field] = val;
  }
}

```

```
function addGradeScaleRow() {
  if (!window.App.settings.gradeScale) window.App.settings.gradeScale = [];
  window.App.settings.gradeScale.push({
    grade: 'G' + (window.App.settings.gradeScale.length + 1),
    points: 7,
    description: 'Custom Grade'
  });
  renderGradeScaleTable();
}

function removeGradeScaleRow(index) {
  window.App.settings.gradeScale.splice(index, 1);
  renderGradeScaleTable();
}

async function saveSettings() {
  const threshold = parseFloat(document.getElementById('settingMinThreshold').value) || 75;
  const margin = parseFloat(document.getElementById('settingWarningMargin').value) || 5;
  const rule = document.getElementById('settingInstitutionRule').value;
  const gradeScale = window.App.settings.gradeScale;

  if (threshold < 50 || threshold > 100) {
    showToast('Attendance threshold must be between 50% and 100%', 'error');
    return;
  }

  const payload = {
    minAttendanceThreshold: threshold,
    warningMargin: margin,
    institutionRule: rule,
    gradeScale
  };

  if (window.App.isBackendConnected) {
    try {
      const res = await fetch(`${API_BASE}/settings`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(payload)
      });
      if (res.ok) {
        window.App.settings = payload;
        saveToLocalStorage();
        renderDashboard();
        if (window.renderAttendanceTable) window.renderAttendanceTable();
        if (window.renderGpaTable) window.renderGpaTable();
        showToast('Settings saved to server database!', 'success');
        return;
      }
    } catch (e) {
      console.warn('Backend settings save failed:', e);
    }
  }

  window.App.settings = payload;
  saveToLocalStorage();
  renderDashboard();
  if (window.renderAttendanceTable) window.renderAttendanceTable();
  if (window.renderGpaTable) window.renderGpaTable();
  showToast('Settings saved locally!', 'success');
}

// ==================== RESTORE / IMPORT JSON BACKUP ====================

function importDataFile(event) {
  const file = event.target.files[0];
  if (!file) return;

  const reader = new FileReader();
  reader.onload = function(e) {
    try {
      const parsed = JSON.parse(e.target.result);
      const data = parsed.data || parsed;

      if (!data.attendance && !data.settings && !data.gpaCourses) {
        showToast('Invalid backup file format.', 'error');
        return;
      }
```

```
      openConfirmModal(
        'Restore Backup Snapshot',
        'Restoring this file will overwrite your existing attendance, syllabus, and GPA records. Do you wish to continue?',
        async () => {
          if (window.App.isBackendConnected) {
            try {
              const res = await fetch(`${API_BASE}/import`, {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ data })
              });
              if (res.ok) {
                await fetchAllFromBackend();
                renderDashboard();
                renderSettingsForm();
                showToast('All data successfully restored from file!', 'success');
                return;
              }
            } catch (err) {
              console.warn('Backend restore failed:', err);
            }
          }

          // Local restore
          window.App.settings = data.settings || window.App.settings;
          window.App.attendance = data.attendance || [];
          window.App.syllabus = data.syllabus || [];
          window.App.gpaCourses = data.gpaCourses || [];
          window.App.cgpaSemesters = data.cgpaSemesters || [];

          saveToLocalStorage();
          renderDashboard();
          renderSettingsForm();
          showToast('Data restored locally from backup!', 'success');
        }
      );
    } catch (err) {
      showToast('Could not parse JSON backup file.', 'error');
    }
  };
  reader.readAsText(file);
}

// ==================== RESET DATABASE ====================

function confirmResetDatabase() {
  openConfirmModal(
    'Reset to Demo State',
    'Are you sure you want to restore the default sample dataset? All custom records will be replaced with standard college demo records.',
    async () => {
      if (window.App.isBackendConnected) {
        try {
          const res = await fetch(`${API_BASE}/reset`, { method: 'POST' });
          if (res.ok) {
            await fetchAllFromBackend();
            renderDashboard();
            renderSettingsForm();
            showToast('Database reset to initial sample state!', 'success');
            return;
          }
        } catch (e) {
          console.warn('Backend reset failed:', e);
        }
      }

      localStorage.removeItem('campus360_data');
      await loadAppData();
      renderDashboard();
      renderSettingsForm();
      showToast('Restored demo records locally!', 'success');
    }
  );
}
```

11. data/database.json

Sample Dataset & JSON Database Schema (145 lines)


```
{
  "settings": {
    "minAttendanceThreshold": 75,
    "warningMargin": 5,
    "institutionRule": "weighted_credits",
    "gradeScale": [
      { "grade": "O", "points": 10, "description": "Outstanding (90-100%)" },
      { "grade": "A+", "points": 9, "description": "Excellent (80-89%)" },
      { "grade": "A", "points": 8, "description": "Very Good (70-79%)" },
      { "grade": "B+", "points": 7, "description": "Good (60-69%)" },
      { "grade": "B", "points": 6, "description": "Above Average (55-59%)" },
      { "grade": "C", "points": 5, "description": "Average (50-54%)" },
      { "grade": "P", "points": 4, "description": "Pass (40-49%)" },
      { "grade": "F", "points": 0, "description": "Fail (<40%)" }
    ]
  },
  "attendance": [
    {
      "id": "att_1",
      "code": "CS301",
      "name": "Mathematics III",
      "conducted": 40,
      "attended": 34,
      "faculty": "Dr. Sarah Jenkins",
      "lastUpdated": "2026-10-25"
    },
    {
      "id": "att_2",
      "code": "CS302",
      "name": "Physics & Engineering Electromagnetics",
      "conducted": 36,
      "attended": 25,
      "faculty": "Prof. Alan Turing",
      "lastUpdated": "2026-10-26"
    },
    {
      "id": "att_3",
      "code": "CS303",
      "name": "Programming in C/C++",
      "conducted": 32,
      "attended": 28,
      "faculty": "Dr. Grace Hopper",
      "lastUpdated": "2026-10-27"
    },
    {
      "id": "att_4",
      "code": "CS304",
      "name": "Data Structures & Algorithms",
      "conducted": 28,
      "attended": 22,
      "faculty": "Prof. Donald Knuth",
      "lastUpdated": "2026-10-26"
    },
    {
      "id": "att_5",
      "code": "CS305",
      "name": "Digital Logic Design",
      "conducted": 30,
      "attended": 21,
      "faculty": "Dr. Claude Shannon",
      "lastUpdated": "2026-10-24"
    }
  ],
  "syllabus": [
    {
      "id": "syl_1",
      "subjectCode": "CS301",
      "subjectName": "Mathematics III",
      "totalUnits": 5,
      "units": [
        { "unitNo": 1, "title": "Linear Algebra & Vector Spaces", "topicsTotal": 6, "topicsCompleted": 6,
"status": "Completed" },
        { "unitNo": 2, "title": "Calculus & Multivariable Functions", "topicsTotal": 8, "topicsCompleted":
8, "status": "Completed" },
        { "unitNo": 3, "title": "Differential Equations & Laplace Transforms", "topicsTotal": 7,
```

"topicsCompleted": 4, "status": "In Progress" },

```
        { "unitNo": 4, "title": "Complex Analysis & Mapping", "topicsTotal": 6, "topicsCompleted": 0,
"status": "Not Started" },
        { "unitNo": 5, "title": "Numerical Methods & Iterations", "topicsTotal": 5, "topicsCompleted": 0,
"status": "Not Started" }
      ]
    },
    {
      "id": "syl_2",
      "subjectCode": "CS302",
      "subjectName": "Physics & Engineering Electromagnetics",
      "totalUnits": 4,
      "units": [
        { "unitNo": 1, "title": "Electromagnetic Fields & Maxwell's Equations", "topicsTotal": 6,
"topicsCompleted": 6, "status": "Completed" },
        { "unitNo": 2, "title": "Optics & Wave Propagation", "topicsTotal": 6, "topicsCompleted": 3,
"status": "In Progress" },
        { "unitNo": 3, "title": "Quantum Mechanics Fundamentals", "topicsTotal": 5, "topicsCompleted": 0,
"status": "Not Started" },
        { "unitNo": 4, "title": "Thermodynamics & Statistical Physics", "topicsTotal": 5,
"topicsCompleted": 0, "status": "Not Started" }
      ]
    },
    {
      "id": "syl_3",
      "subjectCode": "CS303",
      "subjectName": "Programming in C/C++",
      "totalUnits": 5,
      "units": [
        { "unitNo": 1, "title": "Language Basics, Flow & Functions", "topicsTotal": 5, "topicsCompleted":
5, "status": "Completed" },
        { "unitNo": 2, "title": "Pointers, References & Memory Management", "topicsTotal": 6,
"topicsCompleted": 6, "status": "Completed" },
        { "unitNo": 3, "title": "Object Oriented Design & Classes", "topicsTotal": 7, "topicsCompleted": 7,
"status": "Completed" },
        { "unitNo": 4, "title": "File Handling & Stream I/O", "topicsTotal": 4, "topicsCompleted": 2,
"status": "In Progress" },
        { "unitNo": 5, "title": "Templates, Exception Handling & STL", "topicsTotal": 6, "topicsCompleted":
1, "status": "In Progress" }
      ]
    },
    {
      "id": "syl_4",
      "subjectCode": "CS304",
      "subjectName": "Data Structures & Algorithms",
      "totalUnits": 5,
      "units": [
        { "unitNo": 1, "title": "Arrays, Stacks & Complexity Analysis", "topicsTotal": 5,
"topicsCompleted": 5, "status": "Completed" },
        { "unitNo": 2, "title": "Queues, Deques & Linked Lists", "topicsTotal": 6, "topicsCompleted": 6,
"status": "Completed" },
        { "unitNo": 3, "title": "Non-linear Structures: Trees & BSTs", "topicsTotal": 7, "topicsCompleted":
2, "status": "In Progress" },
        { "unitNo": 4, "title": "Graphs & Minimum Spanning Trees", "topicsTotal": 6, "topicsCompleted": 0,
"status": "Not Started" },
        { "unitNo": 5, "title": "Sorting, Searching & Hash Tables", "topicsTotal": 6, "topicsCompleted": 0,
"status": "Not Started" }
      ]
    },
    {
      "id": "syl_5",
      "subjectCode": "CS305",
      "subjectName": "Digital Logic Design",
      "totalUnits": 4,
      "units": [
        { "unitNo": 1, "title": "Boolean Algebra & Logic Gates", "topicsTotal": 5, "topicsCompleted": 5,
"status": "Completed" },
        { "unitNo": 2, "title": "Combinational Logic & Decoders/Multiplexers", "topicsTotal": 6,
"topicsCompleted": 6, "status": "Completed" },
        { "unitNo": 3, "title": "Sequential Logic: Flip-Flops & Counters", "topicsTotal": 6,
"topicsCompleted": 1, "status": "In Progress" },
        { "unitNo": 4, "title": "Memory Units & Programmable Logic (PLDs)", "topicsTotal": 5,
"topicsCompleted": 0, "status": "Not Started" }
      ]
    }
  ],
  "gpaCourses": [
    { "id": "gpa_1", "code": "CS301", "name": "Mathematics III", "credits": 4, "grade": "A", "points": 8 },
    { "id": "gpa_2", "code": "CS302", "name": "Physics & Engineering Electromagnetics", "credits": 4,
"grade": "B+", "points": 7 },
    { "id": "gpa_3", "code": "CS303", "name": "Programming in C/C++", "credits": 3, "grade": "O", "points":
10 },
    { "id": "gpa_4", "code": "CS304", "name": "Data Structures & Algorithms", "credits": 4, "grade": "A+",
"points": 9 },
    { "id": "gpa_5", "code": "CS305", "name": "Digital Logic Design", "credits": 3, "grade": "B", "points":
6 },
    { "id": "gpa_6", "code": "CS306", "name": "Data Structures Laboratory", "credits": 2, "grade": "O",
"points": 10 }
  ],
  "cgpaSemesters": [
    { "id": "sem_1", "semester": "Semester 1", "credits": 20, "sgpa": 8.40 },
    { "id": "sem_2", "semester": "Semester 2", "credits": 22, "sgpa": 8.65 },
    { "id": "sem_3", "semester": "Semester 3", "credits": 21, "sgpa": 8.10 },
    { "id": "sem_4", "semester": "Semester 4 (Current)", "credits": 20, "sgpa": 8.25 }
  ]
}
```

12. README.md

System Architecture & Mathematical Formulas Documentation (125 lines)

```
# Campus360
An evision project

---

## Attendance, Syllabus & GPA/CGPA Tracker Module

A modern, responsive, full-stack student academic tracker built according to the **Campus360 System Architecture & Antigravity IDE Implementation Guide**.
---

## Ø<ß  Key Features

### 1. Ø=ÜÊ Academic Dashboard
- **Overall Attendance Metric**: Automatically calculates cumulative attendance percentage across all enrolled courses.
- **Shortage Alert Banner**: Instant alerts when any course drops below the configured threshold (default 75%), highlighting the exact number of consecutive classes required.
- **Syllabus Completion Gauge**: Real-time progress bar across all subjects and topics for upcoming exams.
- **Semester GPA & Cumulative CGPA Highlights**: Key performance indicators displayed at a glance.
- **Quick Action Buttons**: Fast shortcuts to log attendance, calculate GPA, or plan targets.

### 2. Ø=ÜÅ Subject-Wise Attendance Tracker
- **Fields Stored**: Subject Name, Course Code, Total Classes Conducted, Classes Attended, Faculty Name, Last Updated Date.
- **Input Validation**: Strict validation preventing negative numbers or attended classes exceeding conducted classes.
- **Zero Class Handling**: Displays `Not available yet` rather than division by zero when conducted classes = 0.
- **Status Indicators**:
  - Ø=ßâ **Safe (Green)**: $\ge$ Minimum Threshold (e.g. $\ge 75\%$). Displays the number of **safe bunks** (classes you can safely miss without falling below threshold).
  - Ø=Šá **Near Margin (Yellow)**: Approaching threshold within warning margin (e.g. $70\% - 75\%$).
  - Ø=Ý4 **Shortage (Red)**: Attendance $< 75\%$.
- **Shortage Recovery Formula**:
  $$\text{Classes Needed } (n) = \left\lceil \frac{\text{Threshold} \times \text{Conducted} - 100 \times \text{Attended}}{100 - \text{Threshold}} \right\rceil$$
- **Safe Bunk Formula**:
  $$\text{Safe Bunks } (m) = \left\lfloor \frac{100 \times \text{Attended} - \text{Threshold} \times \text{Conducted}}{\text{Threshold}} \right\rfloor$$
- **Quick Log Controls**: One-click `+1 Present` and `+1 Missed` for rapid timetable updates.
- **Bulk CSV Importer**: Upload any college CSV attendance report, auto-detect columns, map custom headers, preview parsed rows with validation, and confirm before import.

### 3. Ø=ÜÚ Syllabus Completion Tracking
- **Unit-by-Unit Breakdown**: Course units/modules with total sub-topics and completed sub-topics.
- **Visual Progress Bars**: Subject-level and unit-level completion percentages.
- **Status Badges**: `Not Started` ($0\%$), `In Progress` ($1-59\%$), `On Track` ($\ge 60\%$), and `Completed` ($100\%$).
- **Quick Topic Increments**: `+` and `" ` buttons for topic tracking after every lecture.

### 4. Ø>Ýî Semester GPA Calculator (SGPA)
- **Course Rows**: Course Code, Name, Credit Value, Letter Grade, and Grade Points.
- **Configurable Grading Scales**: Integrates directly with institutional grade scales defined in Settings (10-point scale, O=10, A+=9, A=8, B+=7, B=6, C=5, P=4, F=0).
- **Instant Formula Execution**:
  $$\text{SGPA} = \frac{\sum (\text{Course Credits} \times \text{Grade Points})}{\sum \text{Course Credits}}$$
- **Seamless Transfer**: Push calculated SGPA directly into the CGPA Calculator with one click.

### 5. Ø<ß¯ Cumulative CGPA Calculator & Planner
- **Multi-Semester Aggregation**: Calculates cumulative grade point average weighted by semester credits:
  $$\text{CGPA} = \frac{\sum (\text{Semester Credits} \times \text{SGPA})}{\sum \text{Semester Credits}}$$
- **Equivalent Percentage & Classification**: Converts CGPA to percentage and standard degree classification (*First Class with Distinction*, *First Class*, etc.).
- **Target CGPA Planner**: Forecasts what future SGPA is needed in remaining semesters to achieve a graduation target (e.g., aiming for $8.50$ CGPA).

### 6. &™þ  Settings, Persistence & Backup
- **Configurable Minimum Attendance Threshold**: Slider & numeric input (e.g., $75\%$).
- **Warning Buffer**: Configurable margin (default $5\%$).
- **Custom Grade Scale Editor**: Add or edit letter grades and point values.
- **Export & Import**: Full snapshot export/import as JSON, and attendance export as CSV.
- **Reset to Baseline**: One-click restoration of sample college data.
- **Hybrid Storage Architecture**: Automatically persists to backend server REST API database (`data/database.json`), with continuous mirror in browser `localStorage`.

---

## Ø=Þ€ Running the Project

### Prerequisites
- Node.js (v18+ or v24 LTS)
- Web browser (Chrome, Edge, Firefox)

### Start the Backend Server
```powershell
npm start
```
```

```
*Or directly:*
```powershell
node server.js
```

The application will start on **http://localhost:3000**.
Open **http://localhost:3000** in your web browser.

Ø=Ý REST API Endpoints

Method	Endpoint	Description
`GET`	`/api/health`	Service health status
`GET`	`/api/dashboard`	Aggregated metrics, overall attendance %, CGPA, syllabus %
`GET`	`/api/attendance`	Subject-wise attendance list with calculated advice
`POST`	`/api/attendance`	Add new subject record
`PUT`	`/api/attendance/:id`	Update attendance record
`DELETE`	`/api/attendance/:id`	Remove subject record
`POST`	`/api/attendance/:id/increment`	Quick log `present` or `absent`
`POST`	`/api/attendance/bulk`	Bulk import CSV records
`GET`	`/api/syllabus`	Unit-wise syllabus records
`PUT`	`/api/syllabus/:id/unit/:unitNo`	Update unit topic completion
`GET`	`/api/gpa`	Current semester course list & SGPA
`POST`	`/api/gpa`	Save GPA courses
`GET`	`/api/cgpa`	Semesters list & weighted CGPA
`POST`	`/api/cgpa`	Save CGPA semesters
`GET`	`/api/settings`	Read institutional rules & grade scales
`POST`	`/api/settings`	Update settings & grade scales
`GET`	`/api/export`	Export backup (JSON or CSV)
`POST`	`/api/import`	Restore backup from file
`POST`	`/api/reset`	Reset to initial sample data

Ø>Ýê Verification & Manual Testing Checklist

Test Item	Expected Result	Verified
Zero Classes Conducted	Shows `Not available yet` (no NaN or divide-by-zero errors)	'
Attended > Conducted	Form displays error and rejects submission	'
Attendance < 75%	Displays Red Shortage pill + accurate consecutive classes needed	'
Attendance $\ge 75\%$	Displays Green Safe pill + safe bunks count	'
Quick Log Buttons	`+1 Present` and `+1 Missed` update immediately	'
Syllabus Unit Tracker	Topic `+` and `" ` increments recalculate completion %	'
GPA Dynamic Recalc	Changing grade or credits instantly recalculates SGPA	'
CGPA Multi-Semester	Weights by credits $\sum(\text{Credits} \times \text{SGPA}) / \sum \text{Credits}$
Ḃulk CSV Import	Maps custom columns, previews data, validates rows	'
Data Persistence	Refreshing page preserves state from server / localStorage	'
```